

## DAY 4

# Observability, Containers, OpenShift, and CI/CD

Services observable, containerized, and deployed via a completed CI/CD pipeline. · Lab 4 — Deploy the Capstone Services





*"If you cannot see it, you cannot operate it  
— logs, metrics, and traces as one story."*

## MODULE 9

# Observability

Instrument Spring Boot for production: structured logs, Micrometer metrics, distributed tracing, and the alerts that turn a banking incident into a known playbook.



# MONITORING VS. OBSERVABILITY

*Monitoring answers known unknowns with dashboards and thresholds; observability explores unknown unknowns through logs, metrics, and traces.*



## MONITORING: FOCUS ON KNOWN UNKNOWNNS

- Collect metrics -> alert on thresholds -> investigate, resolve, notify
- Best for detecting known issues, SLA tracking, capacity planning, compliance



## OBSERVABILITY: FOCUS ON UNKNOWN UNKNOWNNS

- Collect logs, metrics, traces -> correlate & explore -> understand, decide, improve
- Best for exploring unknown issues, complex behavior, and root cause analysis

**KEY TAKEAWAY** Monitoring answers known unknowns with dashboards and thresholds; observability explores unknown unknowns through logs, metrics, and traces.



# DAY 4 OBJECTIVES AND CAPSTONE PROGRESS



## DAY 4 OBJECTIVES



**Implement Asynchronous Communication**  
Use Kafka to enable event-driven messaging between services.



**Event-Driven Architecture**  
Design and implement domain events and event flows.



**External System Integration**  
Integrate with an external Fraud Detection service via REST API.



**Reliability & Observability**  
Ensure reliability with retries, DLQ, idempotency and monitor with metrics and logs.



**Testing & Validation**  
Write integration and contract tests to validate the complete flow.



## CAPSTONE PROGRESS



**Account Service**  
User and account management

✓ COMPLETE



**Transaction Service**  
Create and manage transactions

✓ COMPLETE



**Notification Service**  
Send alerts and notifications

✓ COMPLETE



**Security & Resilience**  
OAuth2, Resilience4j, Rate Limiting, Config, Observability

✓ COMPLETE



**Event-Driven & Async Communication**  
Kafka setup, events, producers and consumers

🔄 IN PROGRESS



**External Integration (Fraud Service)**  
REST API integration and response handling

🔄 IN PROGRESS



**End-to-End Validation**  
End-to-end flow, tests and monitoring

⌚ PENDING

## CAPSTONE ARCHITECTURE PROGRESSION



Client

DONE



API Gateway

DONE



Account Service

DONE



Transaction Service

DONE



Kafka

IN PROGRESS



Notification Service

DONE



Fraud Detection

IN PROGRESS



Monitoring & Observability

PENDING

# THE THREE PILLARS OF OBSERVABILITY · APPLICATION LOGGING STRATEGY



## LOGS: WHAT HAPPENED?

- Timestamped records of discrete events; high cardinality, unstructured, stored as text
- **Example:** "Error connecting to database at 10:15:32 from service OrderService"



## WHAT TO LOG

- Operational events (startup, requests, business operations)
- Diagnostics (errors, validation failures), security & audit, performance



## METRICS & TRACES

- **Metrics:** numeric measurements over time -- ideal for alerting and dashboards
- **Traces:** follow a request across services -- reveal latency, bottlenecks, dependencies



## HOW TO LOG IT

- Use structured (JSON) logging with consistent log levels
- Include correlation ID, user ID, service, and environment on every entry
- Never log sensitive data -- mask or exclude PII, credentials, tokens

**KEY TAKEAWAY** Logs give detailed context, metrics show trends and health, and traces reveal dependencies and root causes -- each pillar alone is incomplete. • Operational events, diagnostics, security/audit, and performance signals each need a place in a well-defined logging plan.

# FROM WORKING SERVICES TO OPERABLE SERVICES

## WORKING SERVICES

*"It works on my machine"*



### FUNCTIONAL FOCUS

Focus on features and happy path functionality



### MANUAL OPERATIONS

Manual deployment, configuration and troubleshooting



### LIMITED RESILIENCE

Little to no handling of failures, retries or timeouts



### NO OBSERVABILITY

Limited logging, no metrics, no tracing, hard to diagnose



### BASIC TESTING

Unit tests only, limited integration and end-to-end testing



## OPERABLE SERVICES

*"It works in production, for everyone"*



### RELIABILITY BY DESIGN

Built with resilience patterns: retry, circuit breaker, timeout, idempotency



### AUTOMATED OPERATIONS

Externalized configuration, automated deployment and self-service operations



### BUILT-IN RESILIENCE

Graceful handling of failures, fallbacks, bulkheads and rate limiting



### FULL OBSERVABILITY

Logs, metrics, tracing, dashboards and alerting in place



### COMPREHENSIVE TESTING

Unit, integration, contract and end-to-end tests with Testcontainers

## ENABLERS OF OPERABLE SERVICES



### SECURITY

OAuth2, JWT, Input Validation



### RESILIENCE

Resilience4j, Circuit Breaker, Retry, Timeout



### RATE LIMITING

Protect services and upstreams



### CONFIGURATION

Spring Cloud Config, Profiles, Git-backed



### OBSERVABILITY

Actuator, Prometheus, Grafana, Tracing



### TESTING

Integration tests, Testcontainers



### AUTOMATION

CI/CD, Health checks, Runbooks



**OUTCOME:** Reliable, Secure, Scalable and Maintainable Services in Production

# STRUCTURED AND CENTRALIZED LOGGING

*Fluent Bit/Filebeat ship logs from every microservice into Elasticsearch/Logstash/Kibana, where correlationId ties them all back together.*



## CENTRALIZED LOGGING ARCHITECTURE

- Microservices -> Log Shipper/Agent (Fluent Bit/Filebeat) -> Centralized Log Store (Elasticsearch)
- -> **Users & Tools:** search & analyze, dashboards, alerts, audit & compliance



## BEST PRACTICES

- Log in structured JSON consistently; include correlationId/traceld in every log
- Centralize logs, set retention policies, and mask sensitive data

**KEY TAKEAWAY** Fluent Bit/Filebeat ship logs from every microservice into Elasticsearch/Logstash/Kibana, where correlationId ties them all back together.



# STRUCTURED LOGGING AND CORRELATION IDS



Structured logging provides machine-readable logs. Correlation IDs help trace a single request across multiple services.

## WHY STRUCTURED LOGGING?



### Easier Search & Analysis

Structured logs are easy to parse, search and analyze.



### Better Observability

Enables dashboards, alerts and insights.



### Machine Readable

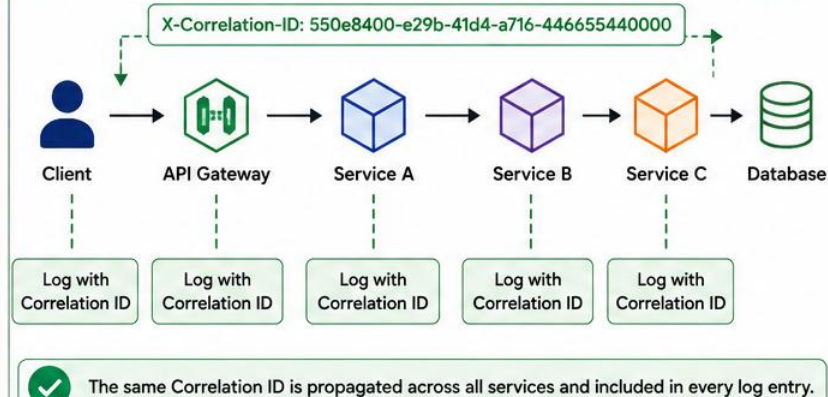
Logs in JSON format are easy for tools to consume.



### Consistent & Standardized

Uniform log structure across all services.

## CORRELATION ID FLOW



## HOW TO IMPLEMENT

- 1 Generate / Extract ID**  
Generate a Correlation ID at the entry point (e.g., API Gateway) or extract from incoming request header.
- 2 Propagate**  
Add the Correlation ID to MDC (Mapped Diagnostic Context) and outgoing requests (e.g., headers).
- 3 Log with ID**  
Include the Correlation ID in every log entry using MDC.
- 4 Clear MDC**  
Clear the MDC after request completion to avoid leakage.

## STRUCTURED LOG EXAMPLE (JSON)

```
{
  "timestamp": "2025-05-15T10:15:30.123Z",
  "level": "INFO",
  "service": "transaction-service",
  "traceId": "550e8400-e29b-41d4-a716-446655440000",
  "spanId": "a1b2c3d4e5f6",
  "message": "Transaction created successfully",
  "http": {
    "method": "POST",
    "path": "/api/transactions",
    "status": 201
  },
  "userId": "user-12345",
  "durationMs": 152,
  "event": "transaction.create"
}
```

## KEY FIELDS TO INCLUDE

|  |                         |                                    |
|--|-------------------------|------------------------------------|
|  | timestamp               | When the log was recorded          |
|  | level                   | Log level (INFO, WARN, ERROR)      |
|  | service                 | Name of the service                |
|  | traceId / correlationId | Unique ID to trace the request     |
|  | spanId                  | (Optional) For distributed tracing |
|  | message                 | Log message                        |
|  | http                    | HTTP method, path, status          |
|  | userId                  | User identifier (if applicable)    |
|  | durationMs              | Request processing time            |
|  | event                   | Business event name                |

## BEST PRACTICES

- Use a standard header name e.g., X-Correlation-ID
- Propagate the ID in all service-to-service calls
- Log in JSON format
- Avoid logging sensitive data
- Clear MDC after request completion
- Use logs for monitoring, alerting and debugging

# METRICS AND TIME-SERIES MONITORING · PROMETHEUS METRICS MODEL · SPRING BOOT ACTUATOR AND MICROMETER



## HOW IT WORKS

- Your Application (Spring Boot) -> Prometheus/Micrometer collection -> Time-Series Storage
- -> Grafana visualization -> Dashboards & Alerts



## KEY METRIC CATEGORIES

- Application performance, resource utilization, business metrics
- Service health, and SLOs and limits (latency/error thresholds)



## CORE CONCEPTS

- **Time series:** a sequence of timestamped values identified by metric name and labels
- Labels give dimensional context; samples are the actual recorded values
- All time series stored in the Prometheus TSDB



## METRIC TYPES

- Counter (monotonically increasing), Gauge (up or down), Histogram/Summary (distributions)



## WHAT THEY PROVIDE

- **Actuator:** production-ready endpoints (health, info, metrics, loggers, threaddump)
- **Micrometer:** a metrics facade that instruments code and publishes to Prometheus, Grafana, Datadog, and more



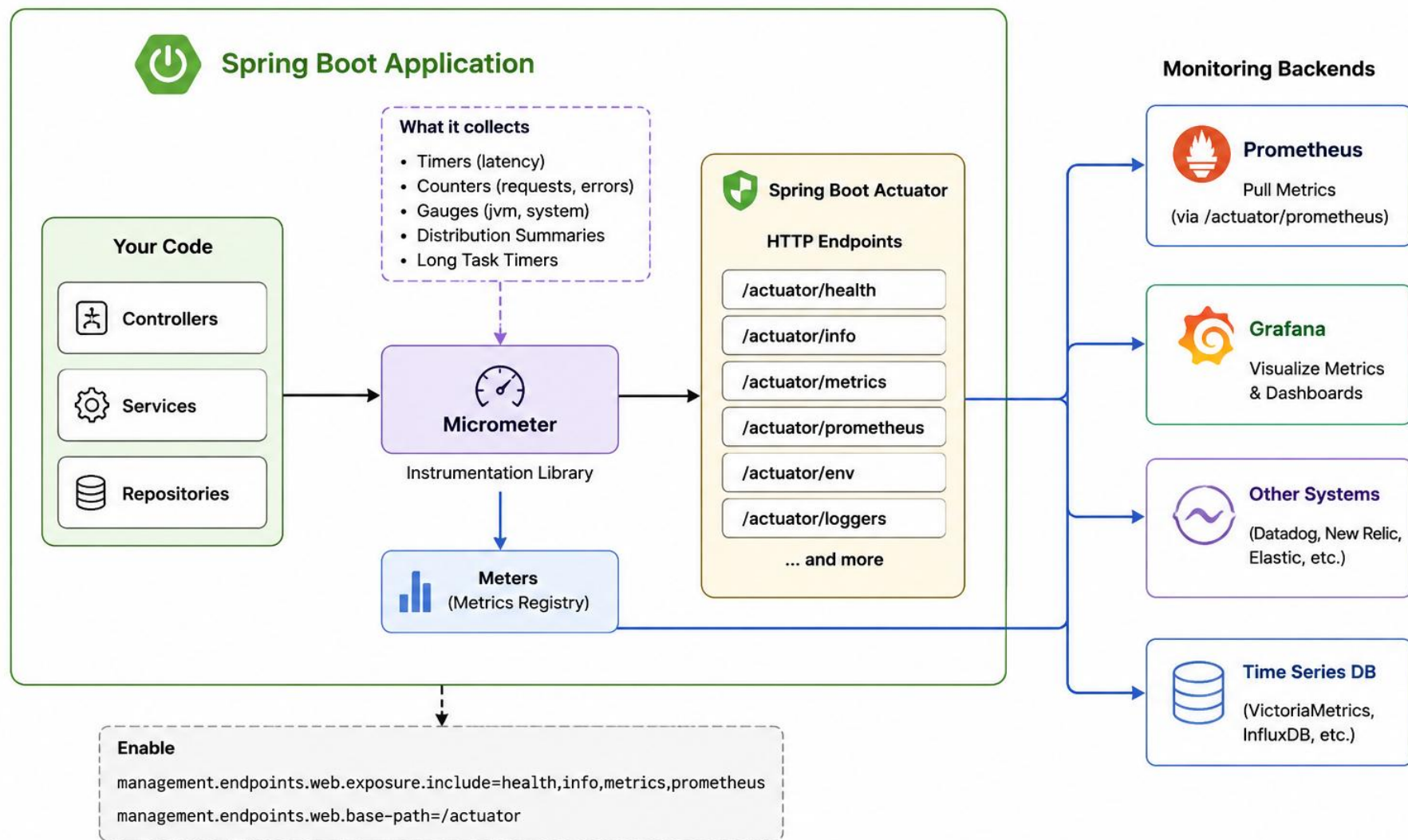
## BEST PRACTICES

- Expose only the endpoints you need; secure Actuator endpoints
- Use meaningful metric names and tags; monitor SLOs with actionable dashboards

**KEY TAKEAWAY** Spring Boot instruments with Micrometer, Prometheus stores the time series, and Grafana turns it into dashboards and alerts.

- `http_requests_total{method="GET", status="200", service="order"} 1027` shows exactly how metric name, labels, and sample value combine.
- `/actuator/health`, `/actuator/metrics`, and `/actuator/prometheus` turn a Spring Boot service into an observable system with two dependencies.

# Actuator and Micrometer



# CUSTOM BUSINESS AND TECHNICAL METRICS · GRAFANA DASHBOARD DESIGN · DISTRIBUTED TRACING



## TYPES OF CUSTOM METRICS

- **Counter:** a monotonically increasing value (e.g., orders.placed)
- **Timer:** records duration and percentiles (e.g., payment.processing.time)
- Gauge and Distribution Summary for point-in-time values and value distributions



## LAYOUT TIPS

- KPIs at the top, trends in the middle (time series), details at the bottom (tables)
- Use variables for environment/service/region filters; group panels into rows



## NAMING & TAGGING GUIDELINES

- Use dot-separated, lowercase, consistent metric names
- **Good tags:** service, region, status, channel; avoid high-cardinality tags like user.id



## WHY DISTRIBUTED TRACING

- End-to-end visibility of a request's full path across services
- Faster root cause analysis; visualizes service dependencies
- Reduces MTTR and validates SLOs and user experience



## DESIGN PRINCIPLES

- Know your audience and focus on the goals/questions the dashboard should answer
- Favor clarity over complexity; use time ranges that fit the use case
- Highlight issues and make insights actionable



## TRACE + METRICS + LOGS = OBSERVABILITY

- Traces (structure) + Metrics (quantitative) + Logs (context) = actionable insight
- Propagate trace-id/span-id across every service and messaging boundary

**KEY TAKEAWAY** A Counter for orders.placed and a Timer for payment.processing.time turn business outcomes and technical health into first-class metrics. • Request rate, error rate, p95 latency, availability, and saturation as top-row KPIs give an instant read on service health before anyone drills in. • A request flowing through API Gateway, Order, Payment, and Inventory services becomes one connected trace, with each hop as a timed span.

# TRACE AND SPAN CONCEPTS · CONTEXT PROPAGATION · OPENTELEMETRY OVERVIEW



## TRACE VS. SPAN

- **Trace:** the complete journey of a request, identified by a unique Trace ID
- **Span:** one unit of work within a trace, with start/end time and its own Span ID
- Spans form a hierarchy -- a span may have a parent span



## SPAN TYPES

- Server span (incoming request), Client span (outgoing request), Internal span (local work)



## HOW CONTEXT FLOWS

- **HTTP/REST:** W3C Trace Context headers (traceparent, tracestate)
- **Messaging (Kafka in Lab 2):** correlation id injected into message headers
- **gRPC and async events:** context carried in metadata or event headers



## THE PROPAGATION LIFECYCLE

1. Create
2. Inject
3. Transmit
4. Extract
5. Continue (new span linked to the trace)



## HOW OPENTELEMETRY WORKS

1. **Instrumentation:** your app/service emits telemetry data via the OTel SDK
2. **Collection:** the OpenTelemetry Collector receives, processes, and exports data
3. **Export:** telemetry is sent to one or more observability backends



## KEY BENEFITS

- Vendor-neutral, standardized APIs and data models, end-to-end visibility
- **Flexible -- use only the signals you need:** traces, metrics, or logs

**KEY TAKEAWAY** Every span carries a trace-id, span-id, parent-span-id, timing, and tags -- the anatomy that lets tools reconstruct the whole request as a tree. • W3C Trace Context headers (traceparent, tracestate) carry the trace-id and span-id across HTTP, messaging, and gRPC boundaries alike. • Instrumentation -> Collection -> Export lets the same OTel SDK send traces, metrics, and logs to Prometheus, Jaeger, Grafana, or Datadog interchangeably.

# CORRELATING LOGS, METRICS, AND TRACES · HEALTH CHECKS AND READINESS SIGNALS



## HOW CORRELATION WORKS

1. A request is made and assigned a unique trace-id
2. The trace-id is propagated across all services
3. Services emit logs, metrics, and spans with the same context
4. Visualization tools (Grafana) link the data using correlation fields and time



## THREE TYPES OF HEALTH SIGNALS

- **Liveness probe:** "Is my app running?" -- failure triggers a container restart
- **Readiness probe:** "Is my app ready to serve traffic?" -- failure removes the pod from Service endpoints
- **Startup probe:** "Is my app started up?" -- protects slow-starting apps during initialization

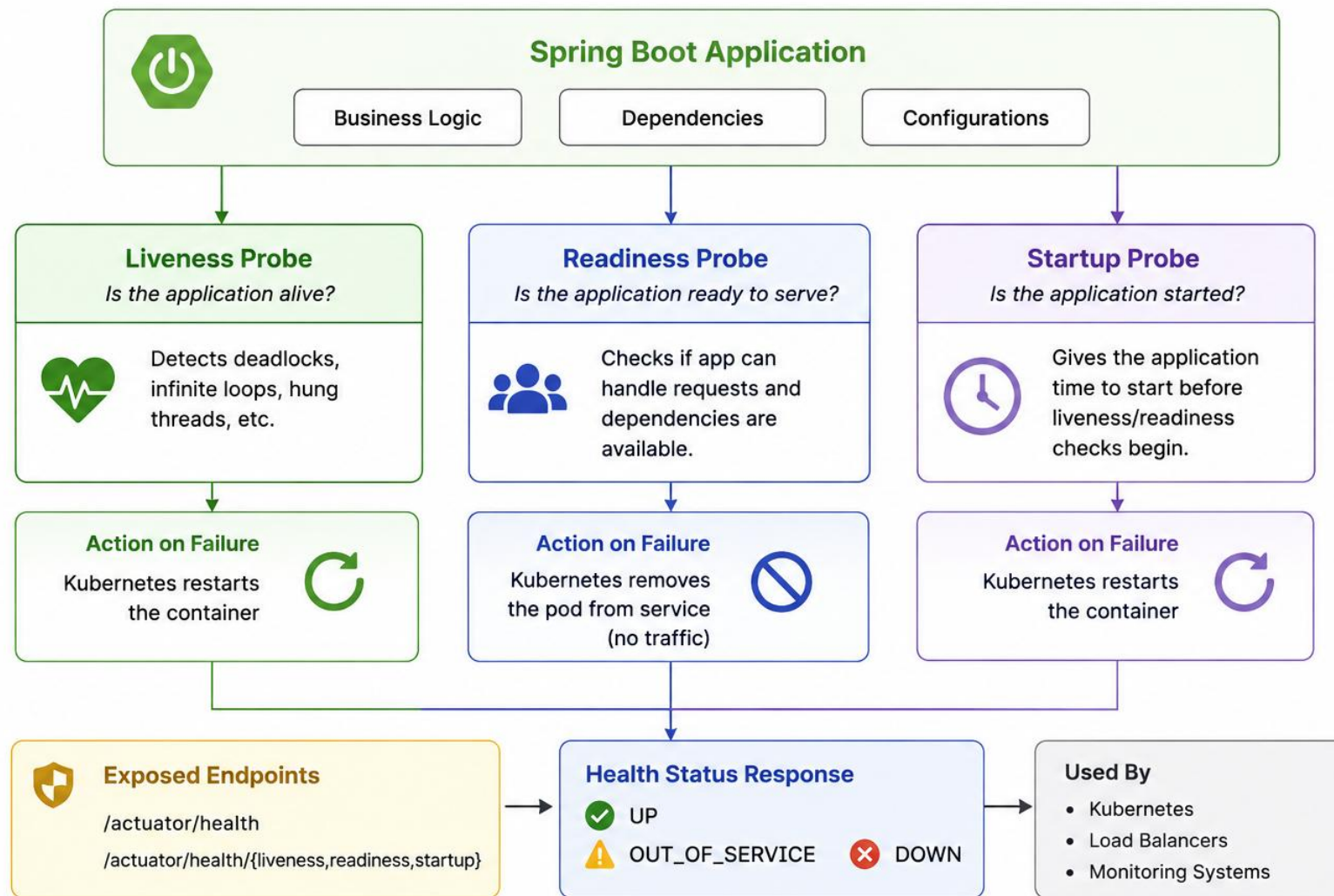


## SPRING BOOT ACTUATOR EXAMPLE

- GET /actuator/health/liveness and /actuator/health/readiness both return {"status": "UP"}

**KEY TAKEAWAY** Jumping from a metrics spike to the exact trace to the precise error log -- all linked by one trace-id -- is what turns signals into a root cause. • Liveness, readiness, and startup probes each answer a different question, and together they keep unhealthy pods off the traffic path.

# Health Checks - Liveness, Readiness, Startup



# LIVENESS VS. READINESS PROBES · ALERTING PRINCIPLES

## LIVENESS VS. READINESS

| ASPECT            | LIVENESS PROBE             | READINESS PROBE                             |
|-------------------|----------------------------|---------------------------------------------|
| Question          | Is my application running? | Is my application ready to handle requests? |
| Action on failure | Restarts the container     | Removes pod from traffic (no restart)       |
| Check speed       | Fast and lightweight       | Can be slower and deeper                    |
| Example           | /actuator/health/liveness  | /actuator/health/readiness                  |

## SIX CORE ALERTING PRINCIPLES

- 1. Be specific
- 2. Be actionable
- 3. Be timely
- 4. Be relevant
- 5. Be noise-free
- 6. Be reviewed

## ALERT FLOW

- Metrics/Logs/Traces -> Detection -> Alert Triggered -> Notification -> Investigation & Action -> Review & Improve

**KEY TAKEAWAY** A liveness failure restarts the container; a readiness failure just stops traffic -- confusing the two can cause unnecessary restarts or bad routing. • An alert tied to user impact or SLO burn, with enough context to act on immediately, is the bar every alerting rule should be held to.

# Metrics Basics



## What is a Metric?

A metric is a numerical measurement captured over time to describe the behavior and health of a system.



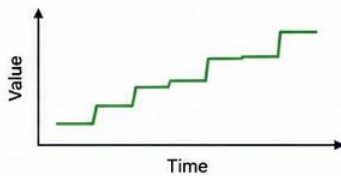
## Key Characteristics

- ✓ Numeric
- ✓ Captured over time
- ✓ Has a Name and Tags (labels)
- ✓ Used for monitoring and alerting

## Metric Types

### 1. Counter

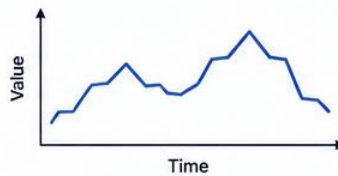
A monotonically increasing value that only goes up.



**Example:**  
http\_requests\_total, errors\_total

### 2. Gauge

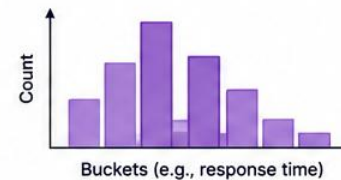
A value that can go up and down.



**Example:**  
jvm\_memory\_used\_bytes,  
cpu\_usage\_percent

### 3. Histogram

Records the distribution of observations into buckets.



**Example:**  
http\_server\_requests\_seconds

### 4. Summary

Similar to Histogram, calculated client-side with quantiles.

| Quantile   | Value (ms) |
|------------|------------|
| 0.50 (P50) | 120        |
| 0.90 (P90) | 250        |
| 0.99 (P99) | 600        |

**Example:**  
http\_server\_requests\_seconds



## Metric Naming Best Practices

- Use clear, descriptive names
- Use consistent naming conventions
- Include unit in the name (e.g., \_seconds, \_bytes, \_total)
- Use labels/tags for dimensions, not in the name

Good: http\_server\_requests\_seconds\_total

Avoid: HTTPRequests, reqTime



## Common Use Cases

- Monitor system health and performance
- Understand usage patterns
- Set alerts and SLOs
- Capacity planning and troubleshooting



## Remember:

Good metrics provide visibility, drive insights, and enable proactive operations.



Application

Emits  
Metrics



**Metrics System**  
Collects, stores,  
and processes  
metrics

Visualize & Alert



**Dashboards / Alerts**  
Charts, dashboards,  
and alerts for  
observability

# REDUCING ALERT FATIGUE

*Focusing on outcomes, deduplicating, and suppressing intelligently turn a wall of noisy alerts into a short list of actionable insights.*



## SIX STRATEGIES TO REDUCE FATIGUE

1. Focus on outcomes
2. Route and escalate
3. Tune and test
4. Deduplicate
5. Suppress intelligently
6. Continuously improve

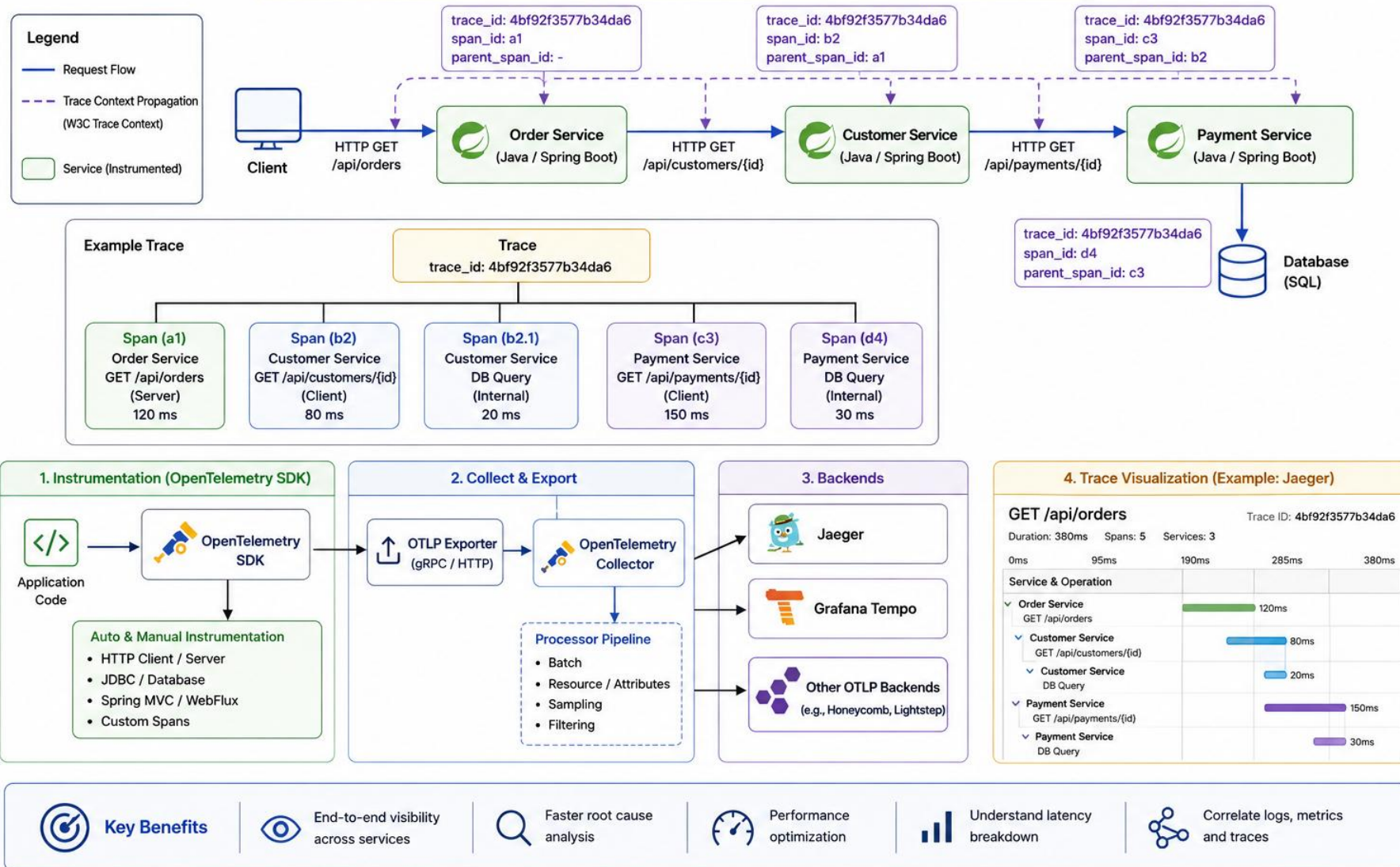


## COMMON CAUSES OF ALERT FATIGUE

- Too many alerts, alerting on symptoms instead of root causes, poorly tuned thresholds
- Duplicate/overlapping alerts and alerts not routed to the right team

**KEY TAKEAWAY** Focusing on outcomes, deduplicating, and suppressing intelligently turn a wall of noisy alerts into a short list of actionable insights.

# Distributed Tracing Demonstration (OpenTelemetry)



# OBSERVABILITY FOR MESSAGING SYSTEMS · OBSERVABILITY MATURITY MODEL · PRODUCTION TROUBLESHOOTING WORKFLOW



## WHAT TO OBSERVE

- Health (broker availability, connectors), throughput (in/out rates)
- Backlog (queue depth, consumer lag), errors (DLQ rate, retries), latency (end-to-end)



## COMMON ALERTS

- Consumer lag exceeds threshold; queue depth or oldest message age too high
- Error rate or DLQ messages detected; broker down or partition unavailable



## THE FIVE MATURITY LEVELS

- **Level 1 Initial:** basic monitoring, manual troubleshooting
- **Level 2 Developing:** centralized logs, dashboards, defined alerting
- **Level 3 Defined:** distributed tracing, SLOs, error budgets, capacity visibility
- **Level 4 Advanced:** anomaly detection, predictive alerts, service dependency maps
- **Level 5 Optimized:** autonomous remediation, observability as a product



## THE SEVEN-STEP WORKFLOW

1. Detect
2. Assess
3. Gather data
4. Analyze
5. Resolve
6. Verify
7. Document & improve



## KEY QUESTIONS TO ASK

- When did the issue start? What changed recently? Who or what is impacted?
- Which service(s) are affected? Is it isolated or widespread?

**KEY TAKEAWAY** Consumer lag and dead-letter-queue depth are the two messaging-specific signals that traditional API monitoring alone would never catch. • Level 1's manual troubleshooting evolves through correlation and prediction all the way to Level 5's autonomous remediation and self-service analytics. • Detect, assess, gather data, analyze, resolve, verify, and document -- each step uses logs, metrics, and traces to find and fix the root cause.

# EXERCISE 4.1 — ADD HEALTH PROBES AND CORRELATION LOGGING

*Module 9 checkpoint A — Operability basics.*



## OBJECTIVE

- Operability basics
- 10 min · code warmup checkpoint
- **Worksheet:** curriculum/day-04/exercises/exercise-4.1-probes-correlation.md



## DO THIS

1. Liveness vs readiness vs startup — which Actuator paths, and why startup exists
2. X-Correlation-Id on the request must appear as correlationId= in logs
3. Health stays public; /actuator/metrics still needs a JWT
4. Tracing (OpenTelemetry) is a demonstration — you do not stand up Jaeger



## EXPECTED RESULT

- You can tell liveness from readiness and find a correlation id in a log line.
- Lab 4 verifies liveness/readiness and correlationId= in logs, then the same probes on OpenShift.

**KEY TAKEAWAY** You can tell liveness from readiness and find a correlation id in a log line.

## MODULE 10

# Containers and Red Hat OpenShift

Package and run services on OpenShift: images, probes, ConfigMaps, routes, and the platform primitives that replace localhost assumptions.



# CONTAINERS VS. VIRTUAL MACHINES

*A VM packages an entire OS plus the app; a container packages just the app and its dependencies, sharing the host kernel underneath.*

## VM VS. CONTAINER

| AREA             | VIRTUAL MACHINE | CONTAINER             |
|------------------|-----------------|-----------------------|
| Includes full OS | Yes             | No                    |
| Startup time     | Slower          | Faster                |
| Resource usage   | Heavier         | Lighter               |
| Packaging        | OS plus app     | App plus dependencies |

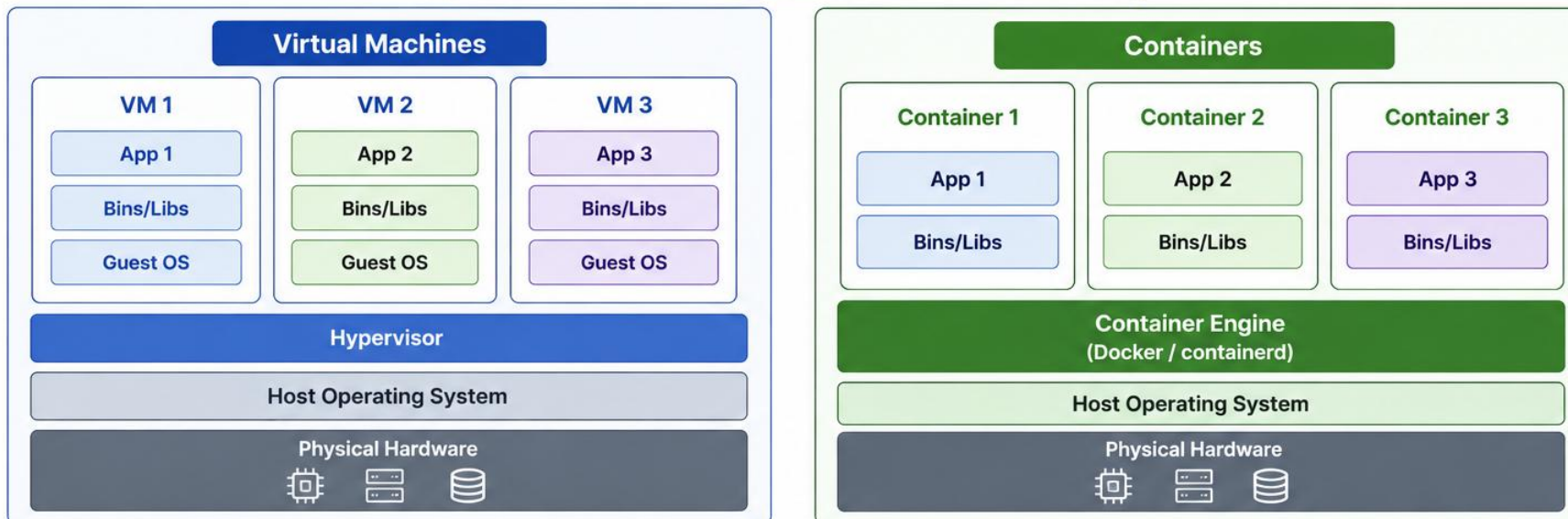


## IMPORTANT CLARIFICATION

- Containers are not automatically secure or production-ready
- Still require secure base images, vulnerability scanning, secrets management, and resource limits

**KEY TAKEAWAY** A VM packages an entire OS plus the app; a container packages just the app and its dependencies, sharing the host kernel underneath.

# Containers vs Virtual Machines



| Aspect                                                                                              | Virtual Machines                            | Containers                              |
|-----------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------------------------|
|  Architecture      | Each VM runs a full Guest OS                | Containers share the Host OS kernel     |
|  Startup Time      | Minutes                                     | Milliseconds to seconds                 |
|  Resource Overhead | High (Guest OS per VM)                      | Low (No Guest OS)                       |
|  Density           | Lower (Fewer VMs per host)                  | Higher (More containers per host)       |
|  Isolation        | Strong (Full OS isolation)                  | Process-level isolation                 |
|  Cost Efficiency | Higher infrastructure cost                  | Lower infrastructure cost               |
|  Management      | Manage OS, patches, VMs                     | Manage containers and images            |
|  Portability     | Lower (Tied to OS and hypervisor)           | Higher (Consistent across environments) |
|  Use Cases       | Legacy apps, different OS, strong isolation | Microservices, cloud-native apps, CI/CD |

# LINUX CONTAINER FUNDAMENTALS · IMAGES, CONTAINERS, AND REGISTRIES · CONTAINER IMAGE LAYERS



## THE TWO KERNEL FEATURES BEHIND CONTAINERS

- **Namespaces:** isolate a process's view of PIDs, network, mounts, users, and hostname
- **Control groups (cgroups):** limit and account for CPU, memory, and I/O usage per process group
- A container runtime (containerd, CRI-O) orchestrates these kernel primitives



## THE THREE CONCEPTS

- **Image:** a packaged template -- `account-service:1.0.0`
- **Container:** a running instance of that image
- **Registry:** stores and distributes images so platforms like OpenShift can pull them



## HOW LAYERING WORKS

- Each instruction (FROM, COPY, RUN) creates a new, cacheable image layer
- Unchanged layers are reused across builds, unchanged parts don't need to rebuild
- Poor layer design creates large images or slow, uncacheable builds

**KEY TAKEAWAY** Under the hood, a container is just a regular Linux process wrapped in kernel namespaces and control groups, not a separate virtualized OS. • `account-service:1.0.0` the image becomes a running Account Service process the moment it's started as a container. • Good layer ordering (least-changing instructions first) lets Docker reuse cached layers instead of rebuilding the entire image every time.

# CONTAINER RUNTIME CONCEPTS · WRITING A DOCKERFILE FOR SPRING BOOT · DOCKERFILE INSTRUCTIONS AND BUILD CONTEXT



## RUNTIME CONCEPTS

- **Container runtime:** the software that creates and manages the container lifecycle on a node
- **CRI (Container Runtime Interface):** the standard API Kubernetes uses to talk to any compliant runtime
- OpenShift uses CRI-O by default -- a lightweight, Kubernetes-native runtime



## COMMON DOCKERFILE INSTRUCTIONS

- FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, USER, ENTRYPOINT, CMD



## BASIC SPRING BOOT DOCKERFILE

- FROM eclipse-temurin:17-jre WORKDIR /app
- COPY target/account-service.jar app.jar EXPOSE 8080
- ENTRYPOINT ["java", "-jar", "app.jar"]



## BUILD CONTEXT DISCIPLINE

- The build context is the file set sent to the image builder
- Use .dockerignore to exclude .git, .idea, \*.log, and other unneeded files

**KEY TAKEAWAY** containerd and CRI-O implement the Container Runtime Interface that Kubernetes and OpenShift use to run pods on every node.

- FROM eclipse-temurin:17-jre, COPY the JAR, EXPOSE 8080, ENTRYPOINT java -jar app.jar -- the minimum viable Spring Boot Dockerfile.
- Without a .dockerignore, .git folders, local credentials, and IDE metadata can silently end up inside the build context and even the image.

# MULTI-STAGE CONTAINER BUILDS · LAYERED SPRING BOOT IMAGES



## MULTI-STAGE CONCEPT

- FROM maven:3.9-eclipse-temurin-17 AS build -> mvn clean package
- FROM eclipse-temurin:17-jre -> COPY --from=build ... app.jar
- The final image contains only the runtime and the compiled JAR, nothing from the build stage

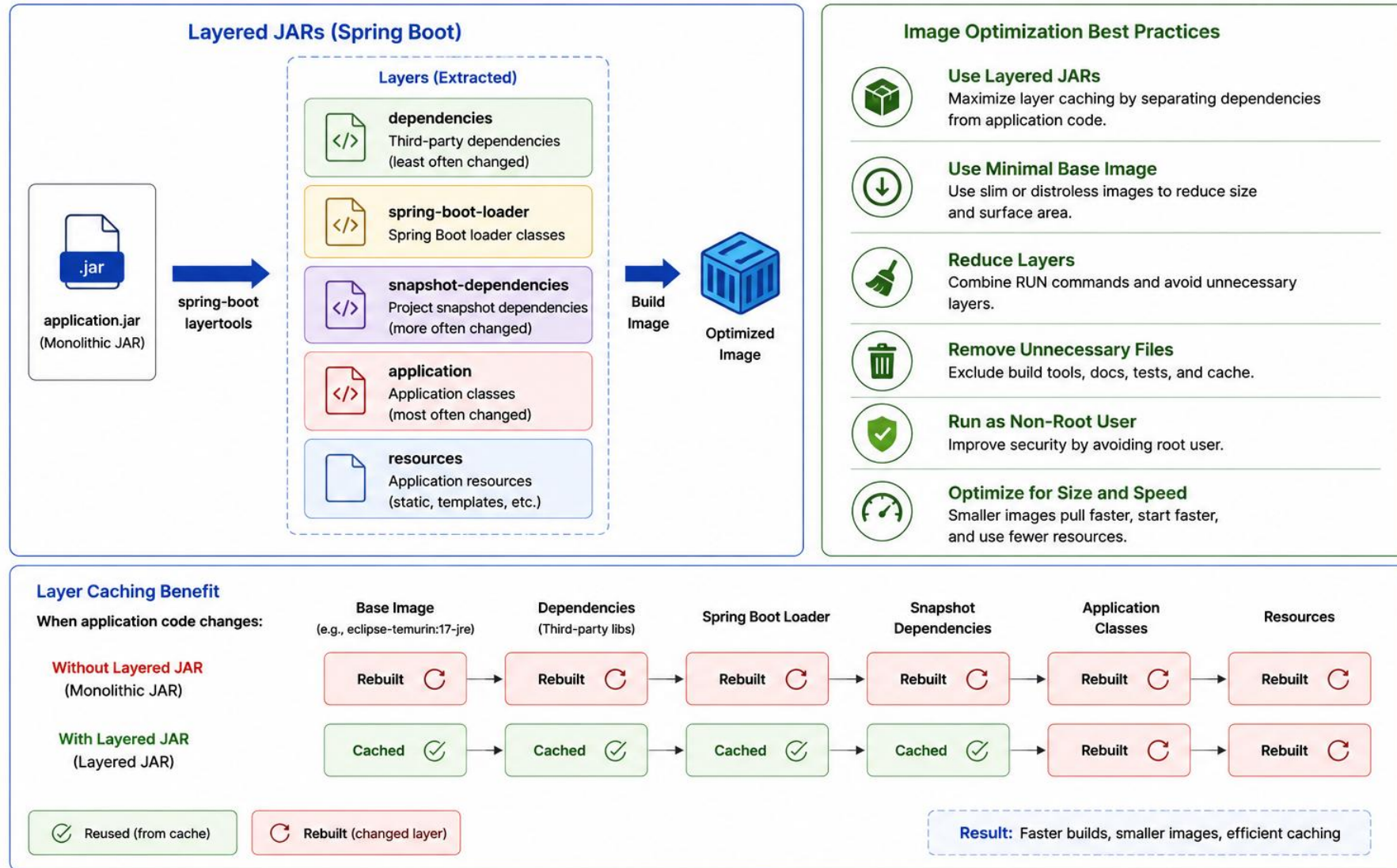


## WHY LAYERED JARS HELP

- spring-boot:build-image or layertools extracts dependencies, resources, and application classes into separate layers
- Dependencies rarely change -- that layer stays cached across most builds
- Only the application-classes layer typically needs to be rebuilt and re-pushed

**KEY TAKEAWAY** The Maven build tools, source code, and intermediate files never make it into the final image -- only the compiled JAR does. • Since dependencies change far less often than your own code, layering means most rebuilds only need to push one small, changed layer.

# Layered JARs and Image Optimization



# BUILDPACKS AND CLOUD-NATIVE BUILDPACKS · CONTAINER IMAGE TAGGING STRATEGIES · CONTAINER REGISTRIES

## WHY BUILDPACKS HELP

- Automatically detect the language/framework and apply best-practice image construction
- No Dockerfile to write or maintain; consistent, reproducible builds across teams
- `mvn spring-boot:build-image` is the common Spring Boot entry point

## COMMON REGISTRIES

- Red Hat Quay, OpenShift internal registry, Docker Hub, GitHub Container Registry, cloud provider registries

## GOOD TAGGING EXAMPLES

- **Semantic version:** 1.4.2
- **Git commit SHA:** a1b2c3d
- **Build number:** build-105
- Avoid relying only on mutable tags like `latest` for traceable deployments

## REGISTRY SECURITY

- Authentication, authorization, image scanning, audit logging, image signing
- Image pull secrets give OpenShift credentials to pull from authenticated registries

**KEY TAKEAWAY** `spring-boot:build-image` uses Paketo buildpacks under the hood, automatically detecting Java and producing a production-ready, layered image. • Semantic versions, Git commit SHAs, and build numbers all answer 'what exactly is deployed?' -- `latest` never does. • Authentication, image scanning, and image pull secrets turn a registry from a simple file store into an enterprise-grade delivery gate.

# IMAGE IMMUTABILITY · CONTAINER CONFIGURATION AND ENVIRONMENT VARIABLES · CONTAINER NETWORKING



## WHY IMMUTABILITY MATTERS

- An image tag that can be overwritten breaks the guarantee that a deployment is reproducible
- Immutable, versioned tags (1.0.0, 1.0.1) make rollbacks and audits reliable
- Pin to a specific digest or version tag in production, never a mutable one



## ENVIRONMENT VARIABLES IN PRACTICE

- `docker run -e SPRING_PROFILES_ACTIVE=prod account-service:1.0.0`
- Supports the build-once, configure-per-environment model
- Spring Boot reads environment variables directly into its configuration properties



## PORT MAPPING EXAMPLE

- Spring Boot listens on port 8080 inside the container
- `docker run -p 8081:8080 account-service:1.0.0` maps that to host port 8081
- The service becomes reachable locally at `http://localhost:8081`

**KEY TAKEAWAY** Immutable, versioned tags guarantee that the same tag always deploys the exact same bits, in every environment, every time.

- `docker run -e SPRING_PROFILES_ACTIVE=prod` passes configuration in without ever touching or rebuilding the image.
- `docker run -p 8081:8080` exposes a service listening on container port 8080 to the outside world on host port 8081.

# CONTAINER STORAGE · CONTAINER RESOURCE LIMITS · RUNNING AS A NON-ROOT USER



## EPHEMERAL VS. PERSISTENT STORAGE

- **Container filesystem:** temporary, lost when the container is removed or replaced
- **Volumes:** external storage mounted into the container, surviving restarts and replacements
- Stateless microservices should avoid depending on local container storage entirely



## WHY RESOURCE LIMITS MATTER

- Without limits, one runaway container can exhaust node CPU or memory
- Requests reserve a guaranteed minimum; limits cap the maximum
- Both should be set deliberately based on real observed workload



## WHY NON-ROOT MATTERS

- Containers should avoid running as root unless explicitly required
- OpenShift enforces security constraints expecting restricted, non-root permissions
- Test images locally as non-root before assuming they'll work on the platform

**KEY TAKEAWAY** Anything written inside a container's own filesystem disappears when it's replaced; only mounted volumes persist across restarts. • CPU and memory requests and limits protect both the container's own stability and the shared cluster it runs on. • A USER instruction in the Dockerfile, matched to OpenShift's security context constraints, avoids surprise permission failures at deploy time.

# Secure Containerfiles and Non-Root Execution

## Secure Containerfile Best Practices

|    |                                                 |                                   |
|----|-------------------------------------------------|-----------------------------------|
| 1  | FROM eclipse-temurin:17-jre-alpine              | Use minimal, trusted base images  |
| 2  |                                                 |                                   |
| 3  | # Create a non-root user                        |                                   |
| 4  | RUN addgroup -S appgroup \                      | Create a dedicated non-root user  |
| 5  | && adduser -S appuser -G appgroup               |                                   |
| 6  |                                                 |                                   |
| 7  | WORKDIR /app                                    | Set working directory             |
| 8  |                                                 |                                   |
| 9  | COPY --chown=appuser:appgroup app.jar ./app.jar | Copy files with correct ownership |
| 10 |                                                 |                                   |
| 11 | USER appuser                                    | Switch to non-root user           |
| 12 |                                                 |                                   |
| 13 | ENTRYPOINT ["java", "-jar", "app.jar"]          | Define endpoint                   |

## Non-Root Execution Benefits

- Improved Security**  
Limits the impact of a container escape or application compromise.
- Principle of Least Privilege**  
Runs the application with only required permissions.
- Reduces Risk**  
Prevents accidental or malicious changes to the container and host system.
- Compliance Friendly**  
Meets security and compliance requirements (e.g., CIS, NIST, PCI DSS).
- Better Isolation**  
Non-root processes are more isolated from the underlying system.

## How Non-Root Execution Works

- 1. Build Image**  
Image includes a non-root user and proper file ownership.
- 2. Run Container**  
Container starts using the specified non-root user.
- 3. Process Runs**  
Application runs with limited privileges inside the container.
- 4. Secure by Default**  
Even if compromised, the attacker has limited access.

Always run containers as non-root users to build secure and resilient applications.

## Additional Security Tips

- ✓ Keep base images up-to-date and scan for vulnerabilities.
- ✓ Use .dockerignore to exclude unnecessary files.
- ✓ Avoid installing unnecessary packages.
- ✓ Drop Linux capabilities if not needed.
- ✓ Set read-only file system when possible (readOnlyRootFilesystem).
- ✓ Use HEALTHCHECK to enable reliable monitoring.
- ✓ Scan images in CI/CD pipeline before deployment.

# CONTAINER HEALTH CHECKS · REDUCING IMAGE SIZE AND ATTACK SURFACE · CONTAINER SECURITY BEST PRACTICES



## CONTAINER HEALTH CHECK BASICS

- A HEALTHCHECK instruction defines a command the runtime periodically executes
- A failing health check can trigger a container restart independent of the orchestration platform
- In OpenShift, this is complemented (not replaced) by liveness/readiness/startup probes



## STRATEGIES TO SHRINK IMAGES

- Use minimal JRE (not JDK) base images for runtime
- Use multi-stage builds so build tools never reach the final image
- Avoid unnecessary package installs and shell utilities in production images



## CONTAINER SECURITY CHECKLIST

- Use minimal, approved base images and scan them for vulnerabilities
- Run as non-root; avoid storing secrets inside images
- Pin base image versions per enterprise policy; keep dependencies minimal

**KEY TAKEAWAY** HEALTHCHECK in a Dockerfile is the container-runtime-level companion to the OpenShift-level liveness and readiness probes covered later. • Minimal JRE base images, multi-stage builds, and removing unnecessary tools all shrink both the download size and the attack surface. • Security isn't one setting -- it's a checklist applied consistently across every image a team builds.

# SUPPLY-CHAIN SECURITY AND SBOMS · TESTING CONTAINERS LOCALLY



## WHY SBOMS MATTER

- An SBOM inventories every dependency and library baked into an image
- **Enables fast vulnerability response:** search the SBOM instead of re-scanning blindly
- Increasingly a compliance and enterprise procurement requirement

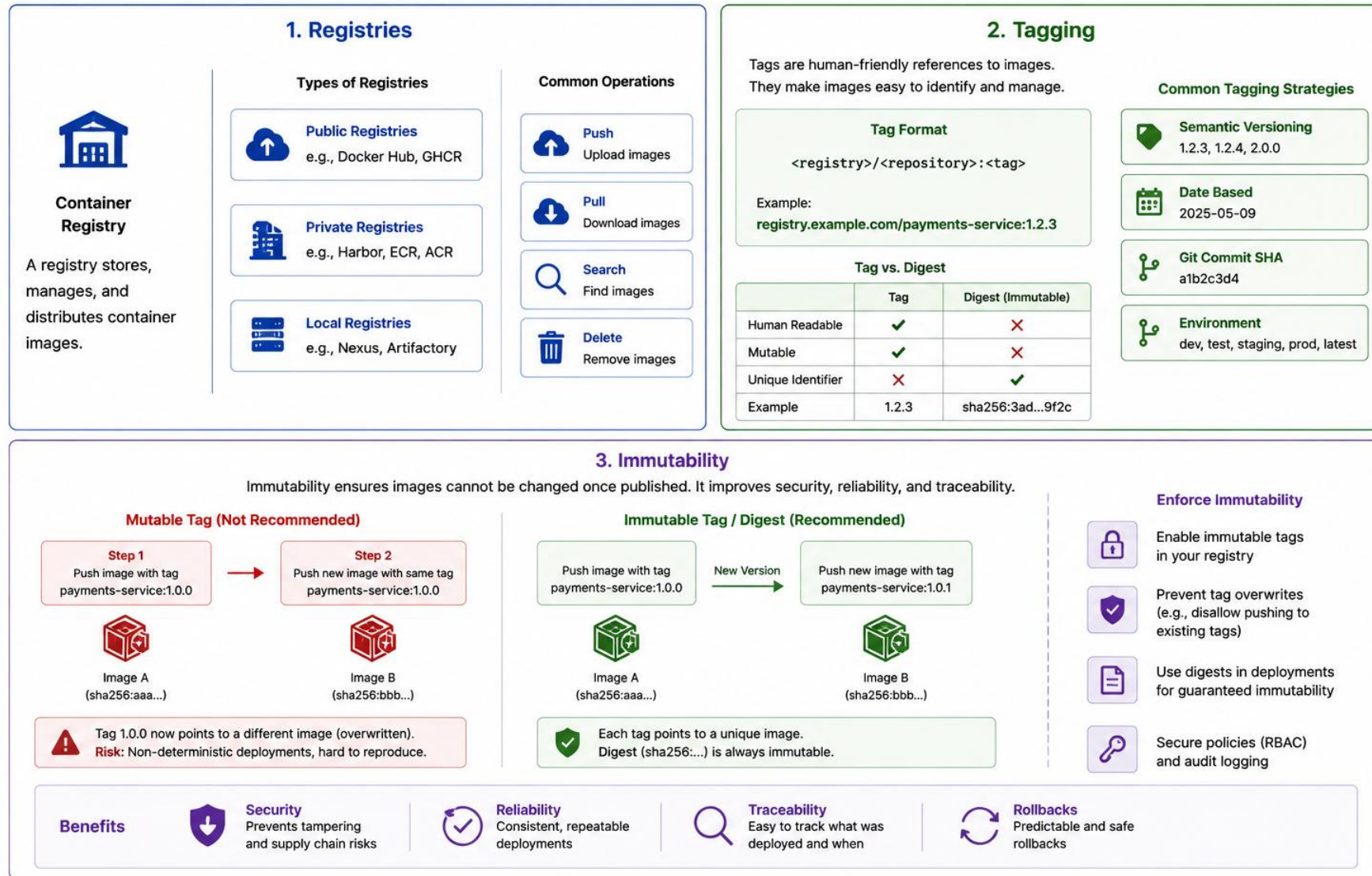


## LOCAL TEST LOOP

1. `docker build -t account-service:1.0.0 .`
2. `docker run -p 8081:8080 account-service:1.0.0`
3. `curl http://localhost:8081/actuator/health` to verify it started correctly

**KEY TAKEAWAY** A Software Bill of Materials turns 'is this image affected by CVE-XXXX' from a guess into a searchable answer. • `docker build`, `docker run -p`, and `curl` the health endpoint form the minimum local test loop before pushing to a registry.

# Registries, Tagging, and Immutability





## CONTAINERIZATION

# Container Troubleshooting Techniques



Identify, diagnose, and resolve container issues quickly to keep your services healthy, reliable, and performant.

### COMMON ISSUES



**Container won't start**  
Misconfiguration, missing dependencies, or errors.



**Crashes / OOM**  
Out of memory, high CPU, or resource limits.



**Networking problems**  
DNS resolution, port mapping, or connectivity issues.



**Volume / storage**  
Permission issues, missing mounts, or full disk.



**Permission denied**  
Running as non-root, file system, or capability issues.



**Slow performance**  
High latency, resource contention, or inefficient app.

### TROUBLESHOOTING WORKFLOW



**1. Identify**  
What is the symptom?



**2. Inspect**  
Check status, logs, and events.



**3. Analyze**  
Find the root cause.



**4. Fix**  
Apply the fix or config change.



**5. Verify**  
Confirm the issue is resolved.

### ESSENTIAL DOCKER COMMANDS

| Purpose              | Command                                                 | Example                                |
|----------------------|---------------------------------------------------------|----------------------------------------|
| List containers      | <code>docker ps -a</code>                               | <code>docker ps -a</code>              |
| Inspect container    | <code>docker inspect &lt;name/id&gt;</code>             | <code>docker inspect my-app</code>     |
| View logs            | <code>docker logs [options] &lt;name&gt;</code>         | <code>docker logs -f my-app</code>     |
| Execute in container | <code>docker exec -it &lt;name&gt; &lt;shell&gt;</code> | <code>docker exec -it my-app sh</code> |
| View resource usage  | <code>docker stats</code>                               | <code>docker stats my-app</code>       |
| Check ports          | <code>docker port &lt;name&gt;</code>                   | <code>docker port my-app</code>        |
| View events          | <code>docker events</code>                              | <code>docker events --since 10m</code> |
| Remove container     | <code>docker rm &lt;name&gt;</code>                     | <code>docker rm my-app</code>          |

### DEEP-DIVE TECHNIQUES

- Check Container Health**  
`docker inspect` to view Health status.
- Resource Analysis**  
Use `docker stats` or cAdvisor.
- Network Diagnostics**  
Test DNS, ping, curl, netstat inside the container.
- File System Inspection**  
Check mounts, disk usage, and permissions.
- Process Inspection**  
Check running processes and resource consumption.
- Reproduce with Minimal Setup**  
Run with minimal config to isolate the issue.

### LOGS & INSPECTION EXAMPLE

```
# View running containers
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  STATUS    PORTS          NAMES
a1b2c3d4e5f6   my-app:1.0 "java -jar app.jar"     Up 2m    0.0.0.0:8080->8080/tcp   my-app

# Follow logs
$ docker logs -f my-app

# Inspect container
$ docker inspect my-app | less

# Execute shell
$ docker exec -it my-app sh

# Check resource usage
$ docker stats my-app
```

### BEST PRACTICES



**Use health checks**  
Detect and recover issues automatically.



**Log to stdout/stderr**  
Centralize logs and monitor effectively.



**Set resource limits**  
Prevent resource exhaustion.



**Keep images minimal**  
Fewer layers, fewer moving parts.



**Test in environments**  
Match production as closely as possible.



**Monitor continuously**  
Alert early and act proactively.

# KUBERNETES AND OPENSIFT OVERVIEW · KUBERNETES VS. RED HAT OPENSIFT · OPENSIFT ARCHITECTURE



## WHAT OPENSIFT PROVIDES

- Container orchestration and deployment automation built on Kubernetes
- Networking, routing, security controls, image management, monitoring
- **Developer workflows:** web console, oc CLI, image streams, builds



## CORE OPENSIFT CONCEPTS

- Cluster, node, pod, deployment, service, route, project
- ConfigMap, secret, image stream, operator



## WHAT OPENSIFT ADDS ON TOP OF KUBERNETES

- Security Context Constraints (SCCs) enforce non-root and least-privilege by default
- Routes (vs. plain Ingress), Image Streams, and BuildConfigs for developer workflows
- An integrated web console, registry, and CI/CD-friendly build pipeline

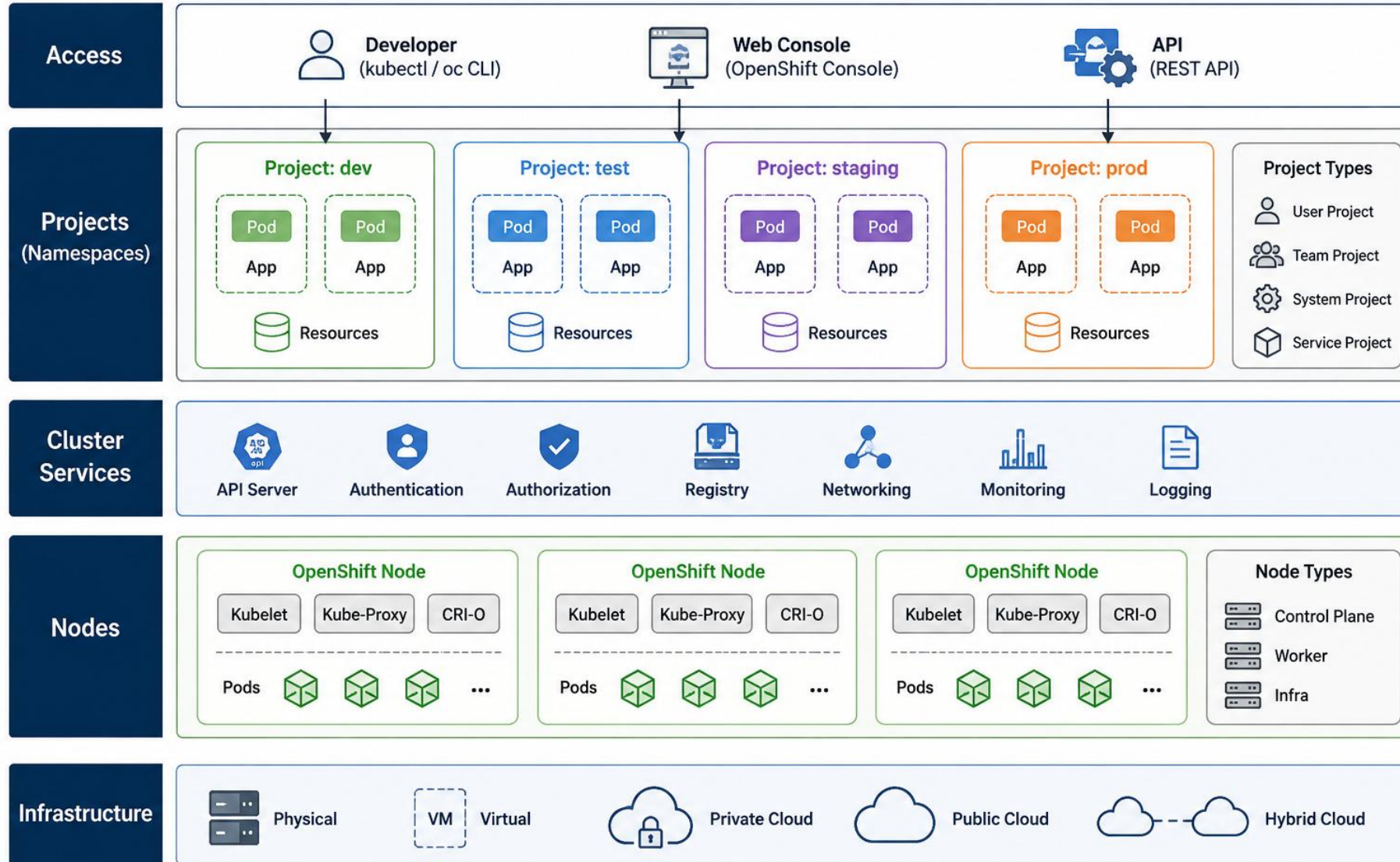


## BANKING EXAMPLE

- Assigned OpenShift project containing Account Service and Transaction Service deployments, each with pods, a service, and a route

**KEY TAKEAWAY** Container orchestration, networking, routing, security, image management, and developer workflows all come bundled with OpenShift. • Vanilla Kubernetes gives you the primitives; OpenShift adds Security Context Constraints, Routes, Image Streams, and BuildConfigs on top. • Pods run on worker nodes, deployments manage pods, services provide stable networking, and routes expose selected services externally.

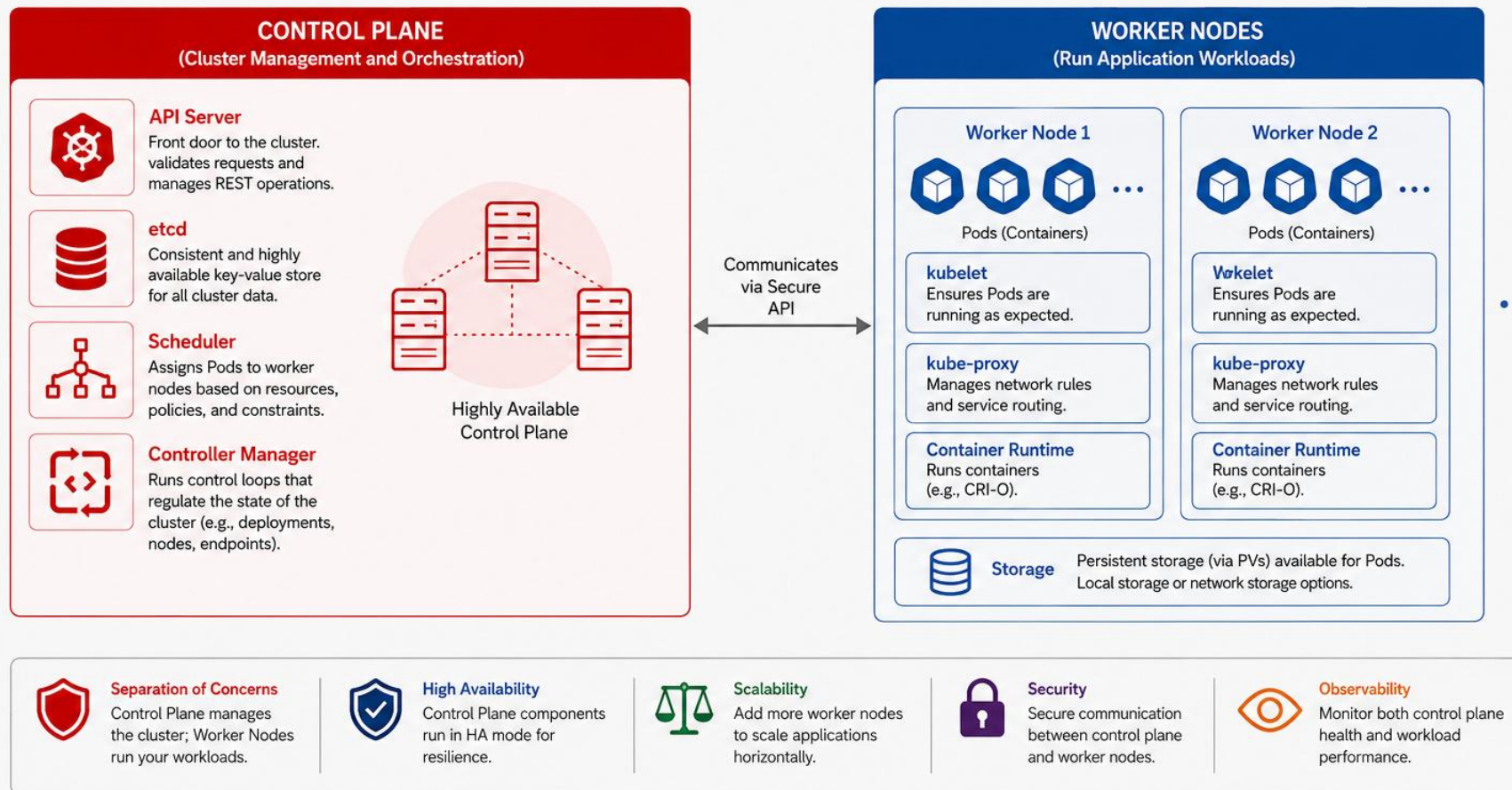
# OpenShift Architecture and Projects



# CONTROL PLANE AND WORKER NODES



OpenShift cluster architecture is built on a clear separation of responsibilities between the Control Plane and the Worker Nodes.



# OPENSIFT PROJECTS AND NAMESPACES · PODS AND REPLICASETS · DEPLOYMENTS AND DEPLOYMENTCONFIGS



## WHY PROJECTS MATTER

- Separate development, test, staging, and production environments
- Support team ownership, access control, and resource quotas
- `oc new-project`, `oc project`, and `oc get` all are the core commands



## WHAT A DEPLOYMENT MANAGES

- Which container image to run, how many replicas, environment variables, rollout strategy
- `oc rollout status` / `history` / `undo` manage and inspect rollouts



## PODS AND REPLICASETS

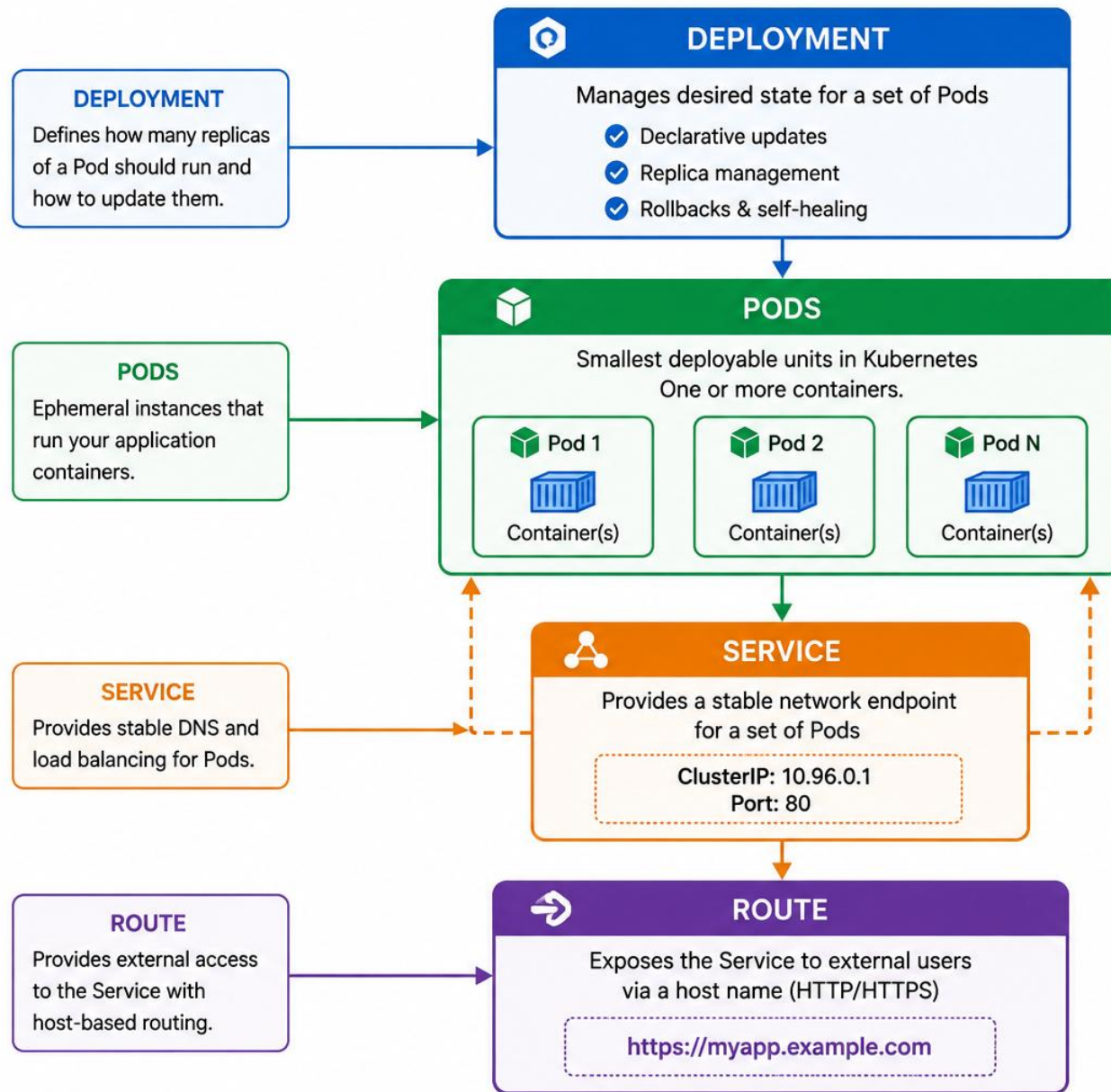
- A pod usually contains one application container, though it can contain more
- A ReplicaSet ensures a specified number of pod replicas are running at any given time
- Deployments manage ReplicaSets, which in turn manage pods



## DEPLOYMENT VS. DEPLOYMENTCONFIG

- **Deployment:** Kubernetes-native, portable, now the recommended default
- **DeploymentConfig:** OpenShift-specific, older, with some OpenShift-only trigger behaviors

**KEY TAKEAWAY** banking-dev, banking-test, banking-staging, and banking-prod each isolate the same services with different configuration and access rules. • If a pod fails, its ReplicaSet notices and creates a replacement automatically -- no manual intervention required. • Kubernetes-native Deployments are now preferred over OpenShift's older DeploymentConfig for most new Spring Boot workloads.



# SERVICES AND INTERNAL NETWORKING · ROUTES AND EXTERNAL ACCESS · CONFIGMAPS AND SECRETS



## HOW SERVICES WORK

- A Service is a stable virtual IP and DNS name that provides reliable access to a set of Pods
- Services use label selectors to route traffic to matching Pods via endpoints
- **Types:** ClusterIP (default, internal-only), NodePort, LoadBalancer, ExternalName



## SERVICE DISCOVERY

- Services register automatically in DNS as `<service-name>.<namespace>.svc.cluster.local`
- Pod-to-pod communication uses pod IPs directly, with no NAT and high performance



## HOW ROUTES WORK

- External Client -> Ingress Controller -> Route -> Service -> Pods
- **Route types:** Edge (TLS terminates at ingress, most common) and Re-encrypt (re-encrypts to the service)



## WHEN TO CREATE A ROUTE

- Public API endpoints, test endpoints, admin UIs, API gateway integration points
- Do not create external Routes for services that should remain internal-only



## CONFIGMAPS: NON-SENSITIVE DATA

- Application configuration properties, feature flags, UI settings, log levels



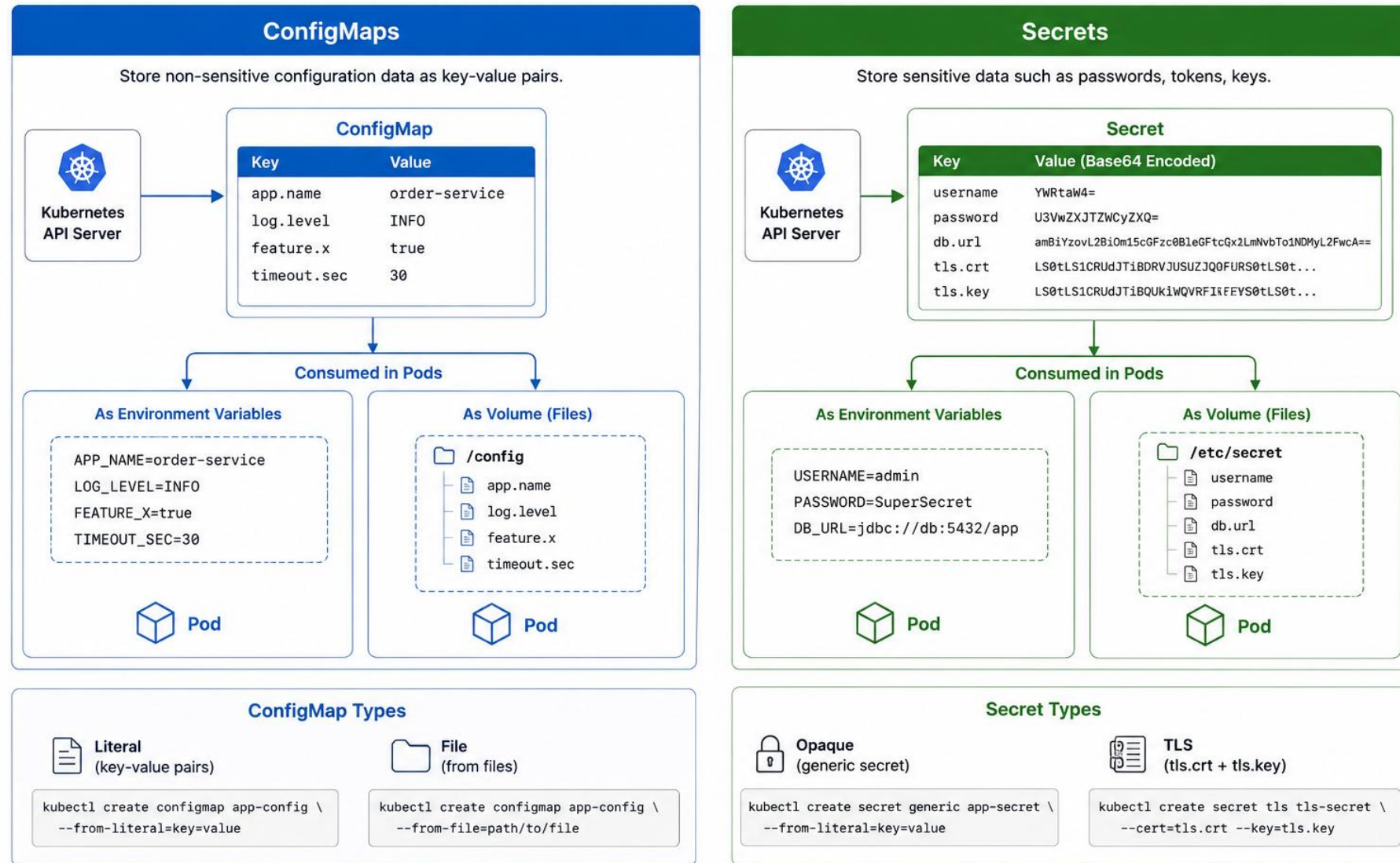
## SECRETS: SENSITIVE DATA

- Database credentials, API keys/tokens, TLS certificates and private keys, OAuth client secrets
- Never store secrets in ConfigMaps; restrict access using RBAC and rotate regularly

**KEY TAKEAWAY** Transaction Service calls `http://account-service:8081`, not a pod IP -- OpenShift load-balances that call across every healthy Account Service pod.

- Edge, re-encrypt, and passthrough TLS termination each trade off where encryption happens between the client and the pod.
- Both can be mounted as files or injected as environment variables -- the only difference is that Secrets are base64-encoded and access-restricted.

# ConfigMaps and Secrets



# PERSISTENT VOLUMES AND CLAIMS · SERVICE ACCOUNTS AND ROLE-BASED ACCESS CONTROL · OPENSIFT IMAGE STREAMS



## KEY CONCEPTS

- **Persistent Volume (PV):** a piece of cluster storage, provisioned by an admin or dynamically
- **Persistent Volume Claim (PVC):** a request for storage with a specific size, access mode, and storage class
- StorageClass defines the type of storage and how new volumes are provisioned



## THE FOUR-STEP RBAC MODEL

1. **Identity:** a Service Account identifies a process running in a pod
2. **RBAC objects:** a Role (namespaced) or ClusterRole (cluster-scoped) defines allowed actions
3. **Binding:** a RoleBinding or ClusterRoleBinding connects the identity to the role
4. **Access:** the bound role grants access to specific resources (pods, secrets, configmaps)



## ACCESS MODES AND RECLAIM POLICIES

- RWO (ReadWriteOnce), ROX (ReadOnlyMany), RWX (ReadWriteMany)
- Retain (keep data), Delete (delete PV and storage), Recycle (deprecated)



## HOW IMAGE STREAMS WORK

- An image stream is a named collection of images pulled from a source registry
- Image stream tags point to specific images (e.g., latest, v1.2.0)
- Tags can be set to automatically track updates in the source registry

**KEY TAKEAWAY** RWO, ROX, and RWX access modes plus Retain/Delete reclaim policies decide who can mount a volume and what happens when a claim is deleted. • Roles define permissions, RoleBindings connect a service account to a role, and the principle of least privilege ties it all together. • Applications reference images by image-stream name instead of a full registry path, decoupling deployments from registry locations.

# BUILDS AND BUILDCONFIGS · OPENSIFT WEB CONSOLE AND CLI



## BUILD STRATEGIES

- **Source-to-Image (S2I):** uses a builder image to assemble the app -- no Dockerfile needed
- **Docker strategy:** uses a Dockerfile for full control over build steps
- **Custom strategy:** runs a custom build script inside the build pod



## WEB CONSOLE

- Browser-based UI to create, view, and manage projects, applications, and resources
- Monitor health, logs, metrics, and events with role-based access built in



## TRIGGERS

- Builds can start manually, on a Git webhook, on image change, or on a schedule



## OC CLI

- Command-line interface for full control and automation
- Ideal for scripting, CI/CD, and bulk actions across a cluster

**KEY TAKEAWAY** Source-to-Image (S2I) turns Java source straight into a runnable image without a Dockerfile, using a builder image as the template. • Use the console for exploration and troubleshooting; use oc for scripting, CI/CD, and any task that needs to be repeatable.

# WORKING WITH THE `OC` COMMAND · DECLARATIVE RESOURCE MANIFESTS · DEPLOYING SPRING BOOT TO OPENSIFT

## COMMON OC COMMANDS

| TASK                | COMMAND                 |
|---------------------|-------------------------|
| List projects       | oc get projects         |
| Switch project      | oc project <name>       |
| List pods           | oc get pods             |
| View logs           | oc logs <pod-name>      |
| Apply configuration | oc apply -f <file.yaml> |

## APPLY AND MANAGE

- oc apply -f my-app.yaml to create or update; oc diff -f to preview changes first

## KEY PRINCIPLES

- Declare, don't imperatively manage -- describe desired state, let the platform enforce it
- **Consistent, repeatable:** manifests are version-controlled, reviewable, and reusable
- **Idempotent:** applying the same manifest multiple times results in the same state

## THE DEPLOYMENT FLOW

1. Prepare app
  2. Build image
  3. Push image
  4. Deploy
  5. Expose
  6. Access
- OpenShift handles scaling, health checks, rolling updates, and high availability automatically

**KEY TAKEAWAY** Tab completion, -n for namespace, and -o wide for extra detail are the quality-of-life habits that make daily oc use fast. • Applying the same manifest twice produces the same result every time -- that idempotency is what makes GitOps and CI/CD pipelines trustworthy. • Source-to-Image, a Docker image, or a BuildConfig are three different paths to the same destination: a running, exposed Spring Boot service.

# CONFIGURING RUNTIME ENVIRONMENTS · RESOURCE REQUESTS AND LIMITS



## THREE CONFIGURATION SOURCES

- **ConfigMaps:** non-sensitive configuration
- **Secrets:** sensitive data (encrypted)
- **Environment variables:** runtime overrides and simple values



## BEST PRACTICES

- Externalize all configuration -- no hard-coded values
- Validate configuration on startup and fail fast when invalid



## QUALITY OF SERVICE CLASSES

- **Guaranteed:** requests == limits for CPU and memory -- highest priority, least likely to be evicted
- **Burstable:** requests < limits -- medium priority
- **BestEffort:** no requests or limits set -- lowest priority, most likely to be evicted

**KEY TAKEAWAY** Same image, different configuration -- the twelve-factor principle that keeps a Spring Boot service portable across every environment. • Guaranteed, Burstable, and BestEffort quality-of-service classes decide which pods get evicted first when a node runs out of resources.

# LIVENESS, READINESS, AND STARTUP PROBES · HORIZONTAL POD AUTOSCALING · ROLLING DEPLOYMENTS



## THE THREE PROBE TYPES

- **Liveness:** is the container running? Failure restarts it
- **Readiness:** is it ready for traffic? Failure removes it from service endpoints
- **Startup:** has the app finished starting? Disables liveness/readiness until it succeeds



## SPRING BOOT EXAMPLE ENDPOINTS

- /actuator/health/liveness and /actuator/health/readiness



## HOW HPA WORKS

- Metrics Server collects CPU/memory/custom metrics from pods
- HPA Controller compares metrics to target values and decides the desired replica count
- The Deployment is adjusted to match -- more pods handle more traffic, fewer save resources



## HOW A ROLLING UPDATE WORKS

- A new pod with the new version is created; an old pod is terminated once the new one is ready
- The process repeats until all old pods are replaced -- traffic is always served



## ROLLBACK

- `oc rollout undo deployment/<name>` reverts to the previous revision with one command

**KEY TAKEAWAY** A startup probe protects a slow-starting Spring Boot app from being killed by liveness checks before it's even finished initializing. • `oc scale` changes replicas manually; a HorizontalPodAutoscaler does the same thing continuously, in response to real load. • `maxUnavailable` and `maxSurge` control exactly how aggressively the rollout proceeds, balancing speed against risk.

# BLUE-GREEN DEPLOYMENTS · CANARY DEPLOYMENTS · ROLLBACKS AND DEPLOYMENT HISTORY



## HOW BLUE-GREEN WORKS

- Deploy the new version (Green) alongside the current one (Blue), fully in parallel
- Validate Green thoroughly, then switch traffic from Blue to Green instantly
- Retire Blue once Green proves stable -- or revert instantly if it doesn't



## HOW ROLLBACKS WORK

- A new version is deployed; monitoring detects errors or performance problems
- Rollback is triggered via CLI, web console, or API; traffic routes back to the last stable version



## HOW CANARY DEPLOYMENT WORKS

- Deploy the new version alongside the stable one, routing only a small percentage of traffic to it
- Monitor metrics, logs, and user feedback; gradually increase traffic if healthy
- Automatic or manual rollback if errors or performance issues are detected



## USEFUL COMMANDS

- `oc rollout history deployment/<name>` to view revisions
- `oc rollout undo deployment/<name> --to-revision=<n>` to roll back to a specific one

**KEY TAKEAWAY** Green gets fully validated with real traffic patterns before the switch, and Blue stays on standby as an immediate rollback target. • 5% traffic to the canary, monitor error rate and latency, then gradually increase to 25%, 50%, and 100% -- or roll back at any step. • `oc rollout undo deployment/<name>` reverts to the last stable revision in seconds, without rebuilding a single container.

# OPENSIFT LOGGING AND MONITORING · TROUBLESHOOTING PODS AND DEPLOYMENTS

## COMMON POD ISSUES

| ISSUE                      | WHAT TO CHECK                          |
|----------------------------|----------------------------------------|
| Pod Pending                | oc describe pod, oc get events         |
| CrashLoopBackOff           | oc logs --previous, oc describe pod    |
| Readiness/Liveness failure | oc describe pod, check probe path/port |
| Volume/Storage issue       | oc describe pod, oc get pvc            |

## WHAT GETS COLLECTED

- Application logs, container/pod logs, cluster and node metrics
- API server and audit logs, events and alerts

## COMMON TOOLS

- Prometheus (metrics), Grafana (dashboards), Elasticsearch (log storage/search), Alertmanager (alert routing)

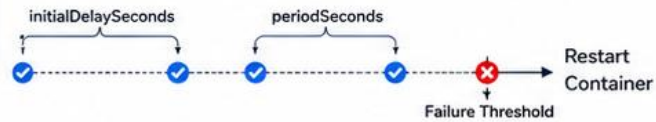
**KEY TAKEAWAY** Fluentd/Fluent Bit ship logs to Elasticsearch, Prometheus scrapes metrics, and Grafana/Kibana turn both into dashboards and alerts. • oc describe pod gives the high-level view first; events almost always contain the most accurate clue about what actually failed.

## Probes



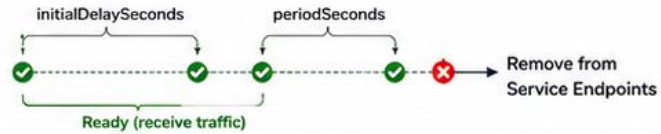
### Liveness Probe

Detects if container is alive.



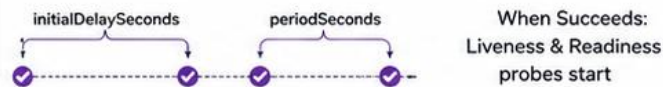
### Readiness Probe

Determines if container is ready to serve traffic.



### Startup Probe

Detects if application has started.



## Resources

### Requests (Minimum guaranteed)



CPU

request: 250m  
(0.25 vCPU)



Memory

request: 256Mi

### Limits (Maximum allowed)



CPU

limit: 1  
(1 vCPU)



Memory

limit: 512Mi

If a container exceeds its limit:



CPU Throttled



Memory OOMKilled

## Scaling

### Horizontal Pod Autoscaler (HPA)



### Manual Scaling

```
$ kubectl scale deploy myapp --replicas=5
```

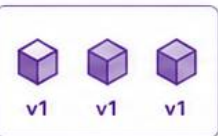
### Vertical Pod Autoscaler (VPA)



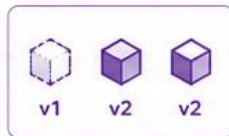
Note: VPA works with appropriate update modes.

## Rollout

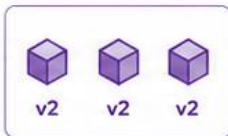
1. Current State  
ReplicaSet: v1  
(3 pods)



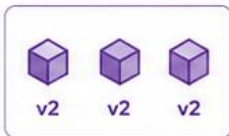
2. Rolling Update  
New ReplicaSet: v2  
created



3. Rolling Update  
Replacing old pods  
gradually



4. Update Complete  
All pods running new  
version



Zero Downtime

### Useful Commands

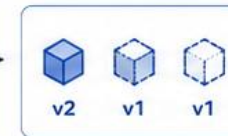
```
kubectl set image deploy/myapp app=v2 | kubectl rollout status deploy/myapp | kubectl rollout history deploy/myapp
```

## Rollback

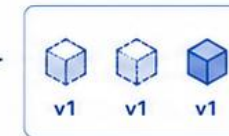
1. Current State  
(v2 running)



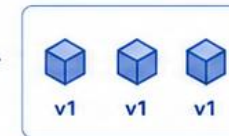
2. Rollback Initiated  
Rolling back to  
previous version (v1)



3. Rolling Back  
Replacing v2 pods  
with v1



4. Rollback Complete  
All pods running  
previous version (v1)



Zero Downtime

### Useful Commands

```
kubectl rollout undo deploy/myapp | kubectl rollout undo deploy/myapp --to-revision=2 | kubectl rollout history deploy/myapp
```

# OPENSIFT SECURITY CONTEXT CONSTRAINTS · OPENSIFT NETWORKING AND NETWORK POLICIES

## COMMON BUILT-IN SCCS

| SCC NAME                    | PURPOSE                                        |
|-----------------------------|------------------------------------------------|
| <b>restricted (default)</b> | Most restrictive: non-root, minimal privileges |
| <b>nonroot</b>              | Must run as non-root UID                       |
| <b>anyuid</b>               | Allows any UID, including root                 |
| <b>privileged</b>           | Full privileges, similar to root on the host   |

## NETWORK POLICY BASICS

- Network Policies define rules for controlling ingress and egress traffic to/from pods
- Applied per namespace and pod selector; if no policy selects a pod, all traffic is allowed
- Ingress rules answer 'who can talk to me?'; egress rules answer 'where can I talk?'

**KEY TAKEAWAY** The default 'restricted' SCC forces non-root, minimal-privilege containers; 'privileged' and 'anyuid' should be reserved for rare, justified cases. • Once a NetworkPolicy selects a pod, only traffic matching its rules is allowed -- everything else is silently denied.

# MANAGING APPLICATION SECRETS · OPENSIFT PRODUCTION- READINESS CHECKLIST

## TYPES OF SECRETS

| TYPE                                  | EXAMPLE USE                                  |
|---------------------------------------|----------------------------------------------|
| <b>Opaque (generic)</b>               | Passwords, tokens, API keys                  |
| <b>kubernetes.io/dockerconfigjson</b> | Registry authentication / image pull secrets |
| <b>kubernetes.io/tls</b>              | TLS termination, mTLS                        |
| <b>Service Account Token</b>          | Pod authentication to the API server         |

## CHECKLIST CATEGORIES (1-5)

1. Application deployment   2. Configuration & secrets   3. Security
4. Networking   5. Storage & persistence

## CHECKLIST CATEGORIES (6-9)

6. Observability   7. Resilience & scalability   8. CI/CD & release
9. Operations

**KEY TAKEAWAY** Opaque, docker registry, TLS, and service-account-token secrets each cover a different sensitive-data use case in OpenShift. • Application deployment, config/secrets, security, networking, storage, observability, resilience, CI/CD, and operations -- all nine, reviewed before go-live.

# EXERCISE 4.2 — CONTAINERIZE ACCOUNT SERVICE

*Module 10 checkpoint B — Secure non-root image.*



## OBJECTIVE

- Secure non-root image
- 10 min · ops warmup checkpoint
- **Worksheet:** curriculum/day-04/exercises/exercise-4.2-containerize-account.md



## DO THIS

- 1. Multi-stage build:** Maven compile in builder, JRE runtime in final image
- 2.** USER md287 (non-root). OpenShift also sets runAsNonRoot: true
- 3.** Immutable tag 1.0.0 — not :latest
- 4.** Images are built from Lab 3 solution context — do not recopy Java trees



## EXPECTED RESULT

- You can explain why the runtime user is not root before you run docker build.
- **Lab 4 Step 3 recap:** multi-stage Containerfile, USER md287, tag md287/account-service:1.0.0.

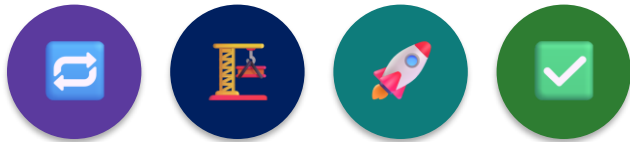
**KEY TAKEAWAY** You can explain why the runtime user is not root before you run docker build.

*"A pipeline you cannot replay is not a pipeline — make every change a deployable artifact."*

## MODULE 11

# DevOps and CI/CD

Move from commit to OpenShift with evidence: CI, image promotion, GitOps, quality gates, and the release discipline a bank expects.



# DEVOPS PRINCIPLES AND CULTURE · DEVOPS LIFECYCLE FOR MICROSERVICES



## CORE DEVOPS PRINCIPLES

- Collaboration across development, operations, security, and quality teams
- Automation of repeated delivery steps to reduce manual error
- Continuous feedback and improvement flowing from monitoring back into planning



## COMMON MISTAKE

- Treating DevOps as only Jenkins or GitHub Actions instead of shared ownership and practice

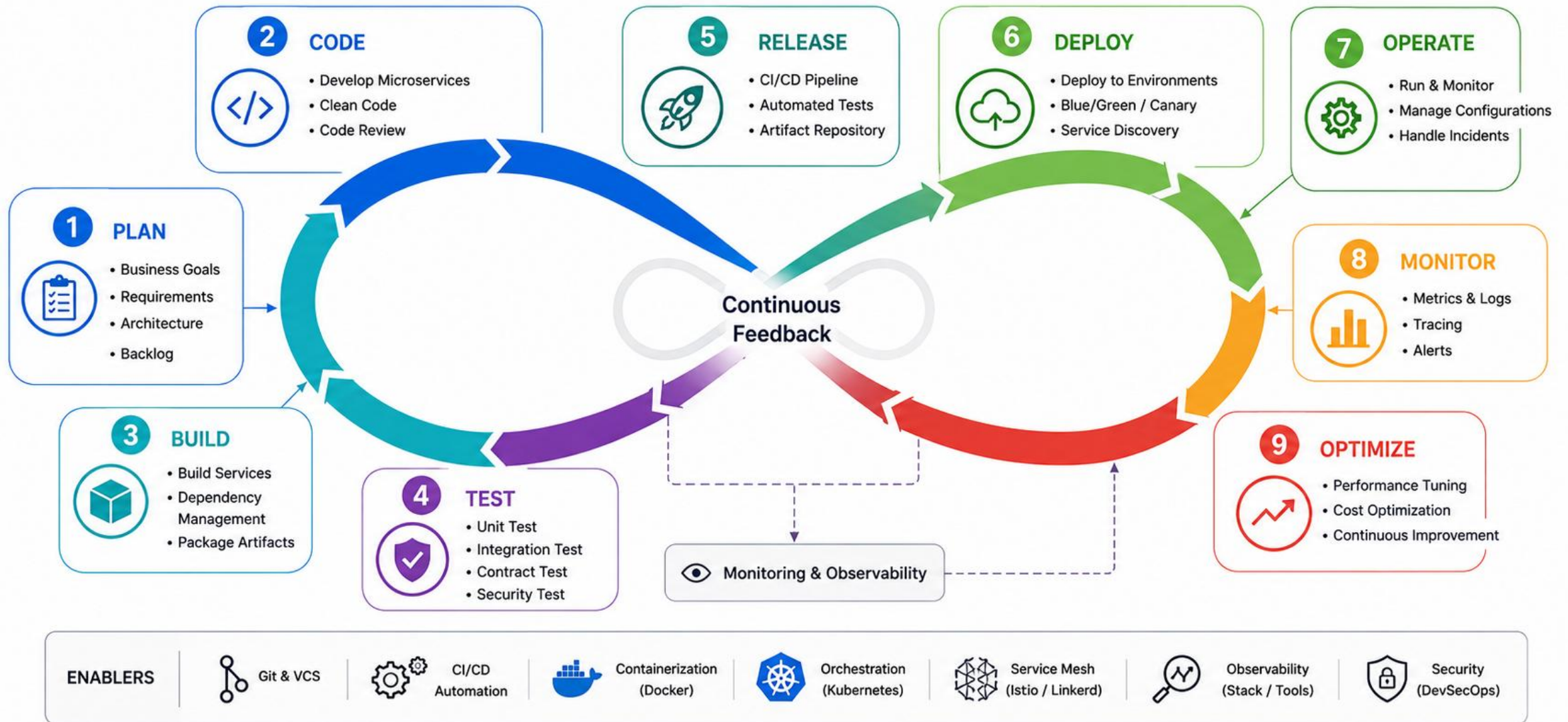


## THE EIGHT LIFECYCLE STAGES

- Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor
- Feedback from monitoring and operations flows back into planning, closing the loop

**KEY TAKEAWAY** Collaboration, automation, and continuous feedback matter more than which specific tools a team adopts. • Account Service moves through this full loop for every feature, with monitoring data shaping the next planning cycle.

# DevOps Lifecycle for Microservices



# CONTINUOUS INTEGRATION · CONTINUOUS DELIVERY VS. CONTINUOUS DEPLOYMENT · PIPELINE AS CODE



## WHAT CI VALIDATES

- Does the code compile? Do tests pass? Does it package successfully?
- Does code quality meet standards? Are dependencies acceptable?



## THE KEY DISTINCTION

- **Continuous Delivery:** automatically build, test, and prepare for deployment -- human approval still required
- **Continuous Deployment:** automatically deploy approved changes to production after passing required checks



## TYPICAL CI STAGES

- Checkout -> Compile -> Unit Tests -> Integration Tests -> Static Analysis -> Package -> Publish Reports

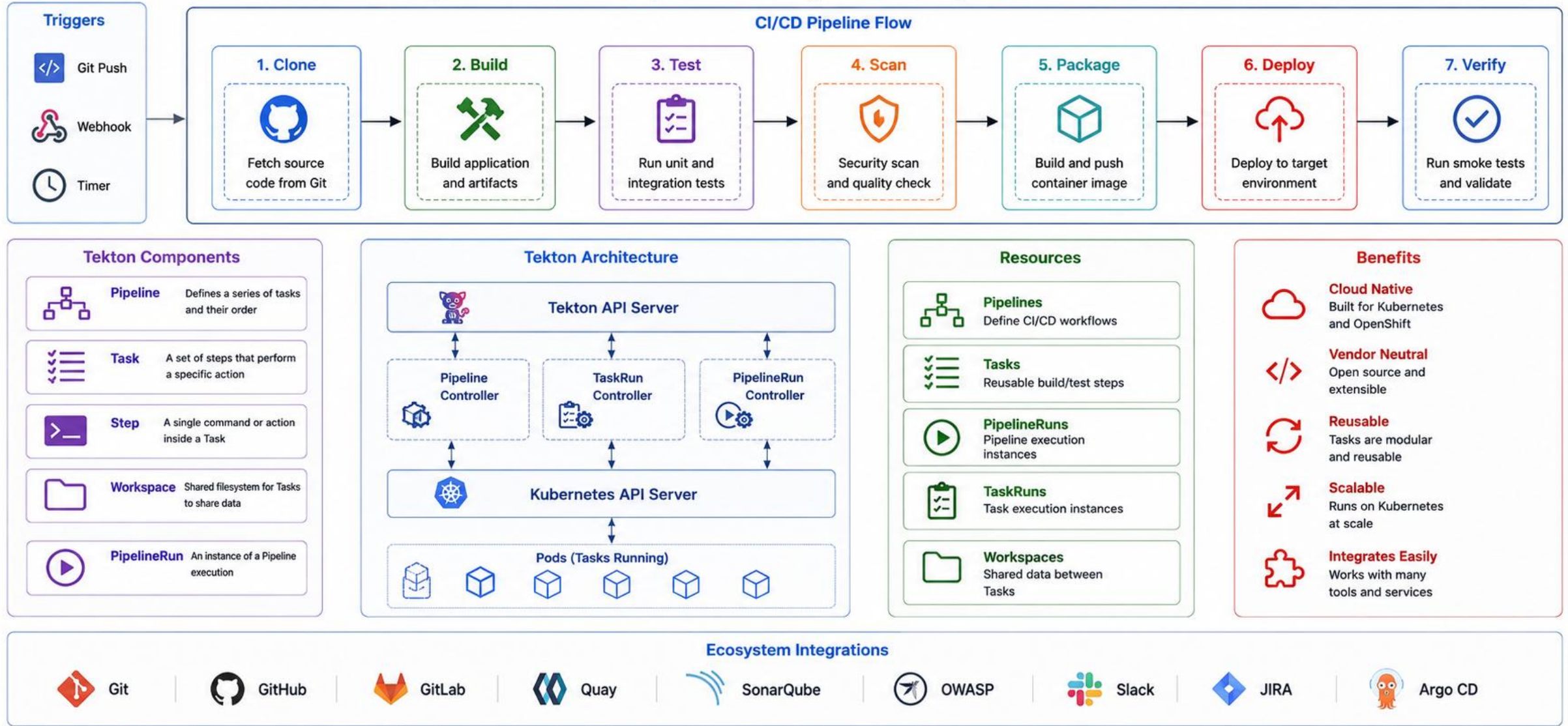


## WHY PIPELINE AS CODE MATTERS

- Instead of manually configuring build steps in a UI, teams store the pipeline definition with the code
- Makes delivery repeatable, reviewable, and version-controlled

**KEY TAKEAWAY** Checkout, compile, unit test, integration test, static analysis, package, publish reports -- the typical CI pipeline shape. • Continuous delivery always leaves a person deciding when to release; continuous deployment ships automatically once checks pass. • Version-controlled pipeline definitions mean delivery logic gets reviewed, diffed, and rolled back exactly like application code.

# OpenShift Pipelines (Tekton) Overview



# SOURCE CONTROL STRATEGY

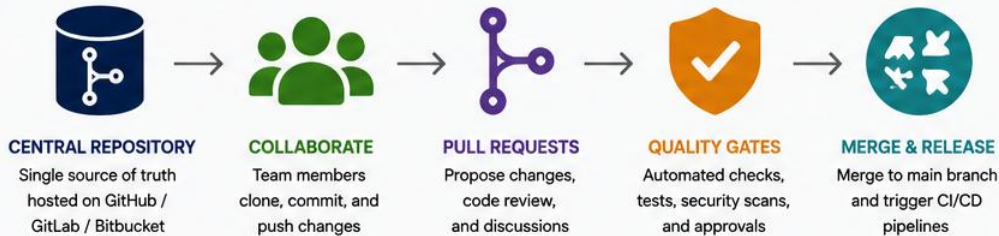
A strong source control strategy provides a single source of truth for code, enables collaboration, traceability, and safe, repeatable delivery.



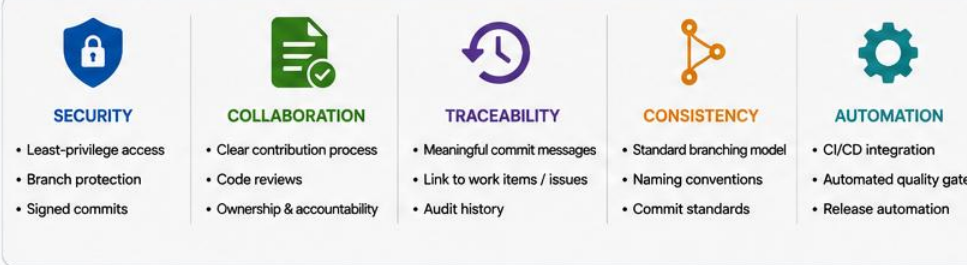
## GOAL

Manage code changes securely, collaborate effectively, and enable reliable delivery.

## STRATEGY ARCHITECTURE



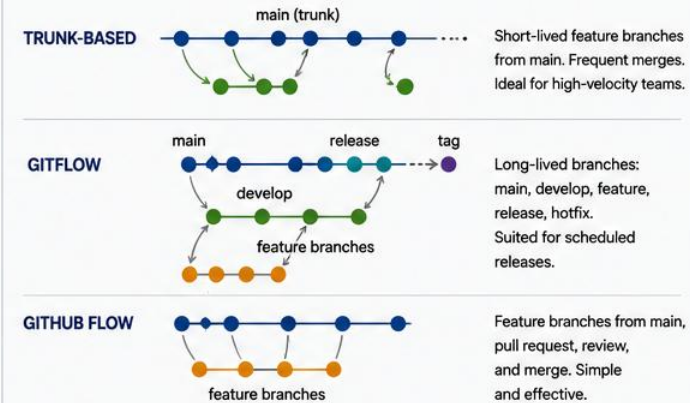
## STRATEGY PILLARS



## KEY BENEFITS



## COMMON BRANCHING MODELS



## BEST PRACTICES

- ✓ Keep the main branch always deployable.
- ✓ Make small, focused commits with clear messages.
- ✓ Review all changes before merging.
- ✓ Automate builds, tests, and quality checks.
- ✓ Protect important branches with required reviews and status checks.
- ✓ Remove stale branches and keep the repository clean.
- ✓ Continuously integrate and deliver value.



Control Your Code. Protect Your Work. Accelerate Your Delivery.

Think Before You Commit. Automate Before You Merge.



Plan



Code



Commit



Review



Merge



Deploy



Monitor

# GIT BRANCHING MODELS · TRUNK-BASED DEVELOPMENT · PULL REQUESTS AND CODE REVIEWS

## THREE BRANCHING MODELS

| MODEL       | BEST FOR                                  |
|-------------|-------------------------------------------|
| Trunk-Based | Fast-moving teams, CI/CD focused          |
| GitFlow     | Enterprise, regulated, scheduled releases |
| GitHub Flow | Simple, CI/CD, open source projects       |

### THE SIX-STEP WORKFLOW

1. Create short-lived branch   2. Develop   3. Pull request   4. Code review
5. Merge to trunk (multiple times a day)   6. Automate & deliver

### WHY IT REDUCES RISK

- Low merge complexity, fast feedback through CI, trunk is always releasable

### PULL REQUEST WORKFLOW

1. Create branch   2. Make changes   3. Open PR   4. Code review   5. Approve   6. Merge

### WHAT GOOD REVIEWS FOCUS ON

- Code correctness, code quality, security, test coverage, documentation, performance

**KEY TAKEAWAY** Trunk-based suits fast-moving CI/CD teams; GitFlow suits regulated, scheduled releases; GitHub Flow suits simple, continuous deployment.

- Small commits, frequent integration, and fast feedback are what let trunk-based teams keep main deployable at every commit.
- Code correctness, quality, security, test coverage, documentation, and performance are the six lenses a good review applies.

# SEMANTIC VERSIONING · ARTIFACT VERSIONING AND TRACEABILITY · BUILD ONCE, DEPLOY MANY



## THE VERSION FORMAT

- **MAJOR:** incompatible API changes
- **MINOR:** backward-compatible new functionality
- **PATCH:** backward-compatible bug fixes



## EXAMPLES

- 1.5.0 adds functionality; 2.0.0 introduces breaking changes; 1.4.3 is a bug fix



## ARTIFACT LIFECYCLE

- Code Commit -> Build -> Publish -> Deploy -> Trace
- Artifacts are immutable and versioned, stored in a secure artifact repository



## WHAT TRACEABILITY ANSWERS

- What ran? Where? When? Built from what code? By which pipeline? With which config?



## BUILD ONCE FLOW

- Code -> Build -> Package -> Verify -> Version & Store -> Deploy to dev, QA, staging, production



## KEY PRINCIPLE

- Artifacts are immutable -- never change, only version. Promote, don't rebuild

**KEY TAKEAWAY** A major bump signals breaking changes, a minor bump signals new backward-compatible functionality, and a patch signals a safe bug fix. • Source code, pipeline run, artifact, container image, deployment, and runtime instance form one unbroken traceability chain. • Promoting one immutable, versioned artifact through dev, QA, staging, and production eliminates 'it works on my machine' problems entirely.

# AUTOMATED QUALITY GATES · STATIC CODE ANALYSIS

## FOUR QUALITY GATES

| GATE                  | WHEN              | PURPOSE                                     |
|-----------------------|-------------------|---------------------------------------------|
| Code Quality Gate     | After commit      | Ensure code is clean, maintainable, secure  |
| Build Quality Gate    | After build       | Ensure the build is successful and reliable |
| Security Quality Gate | After tests       | Ensure the app is secure and compliant      |
| Release Quality Gate  | Before production | Ensure the release is stable and ready      |

## WHAT STATIC ANALYSIS DETECTS

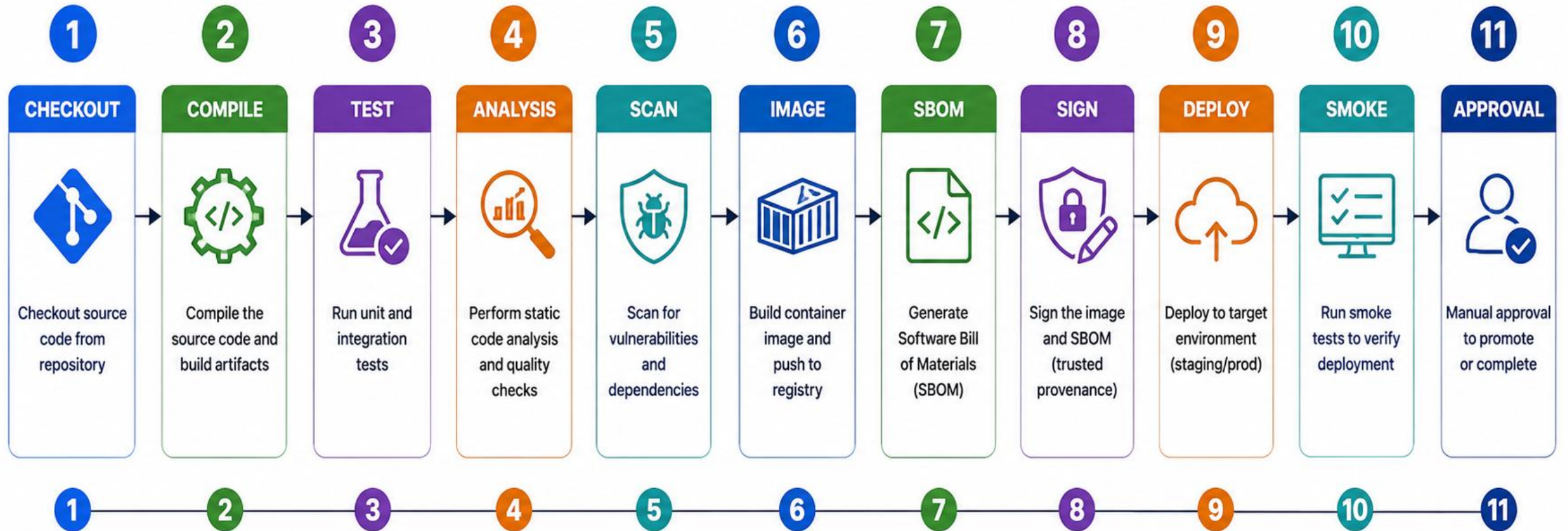
- Security vulnerabilities (SQLi, XSS, hardcoded secrets), bugs, code smells
- Compliance issues against OWASP Top 10, PCI-DSS, HIPAA, and custom rules

## TOOLS

- SonarQube, Checkmarx, Veracode, Fortify SCA, GitHub CodeQL

**KEY TAKEAWAY** Code, build, security, and release quality gates each check a different concern at a different point in the pipeline. • Fixing a defect found in production costs roughly 30x what fixing it at the requirements stage would have cost.

# Pipeline Stages



# SOFTWARE COMPOSITION ANALYSIS · SECRETS SCANNING

## HOW SCA WORKS

1. Identify 2. Analyze 3. Assess 4. Report 5. Remediate

## HOW SECRETS SCANNING WORKS

1. Scan 2. Detect 3. Alert 4. Remediate 5. Prevent

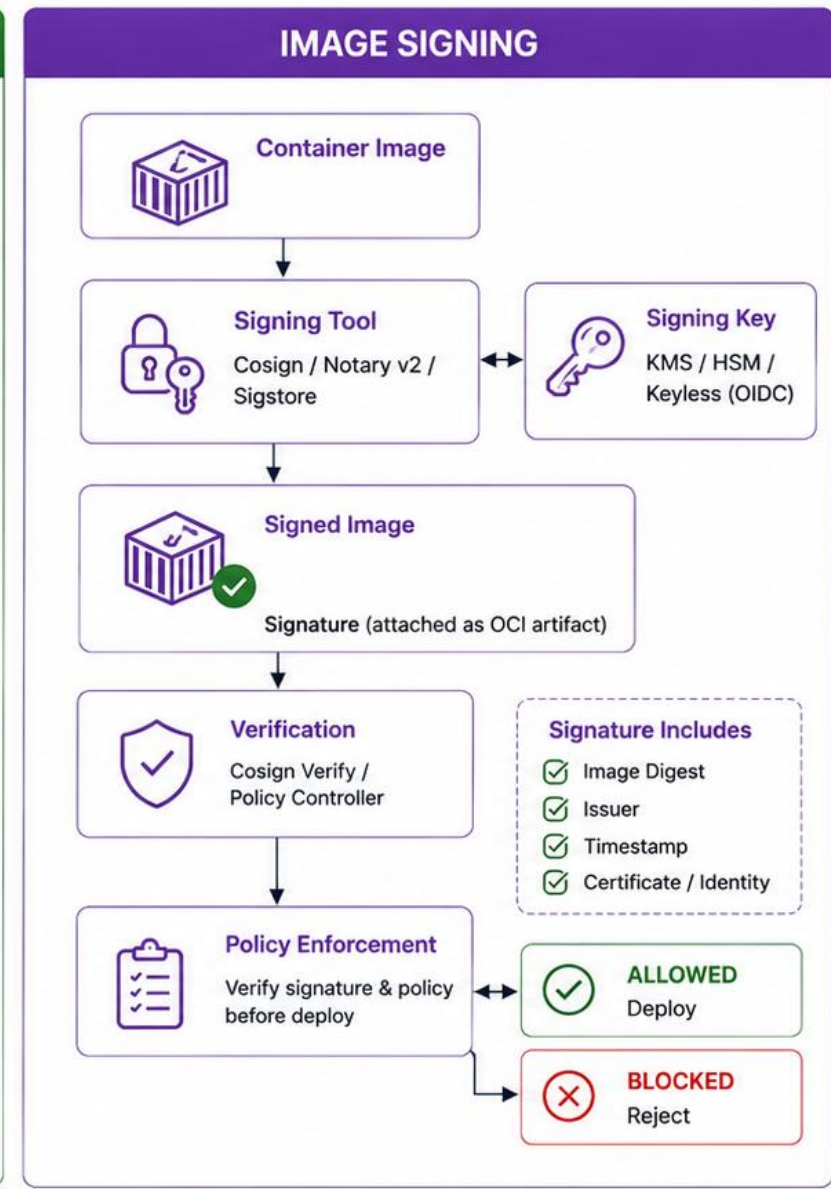
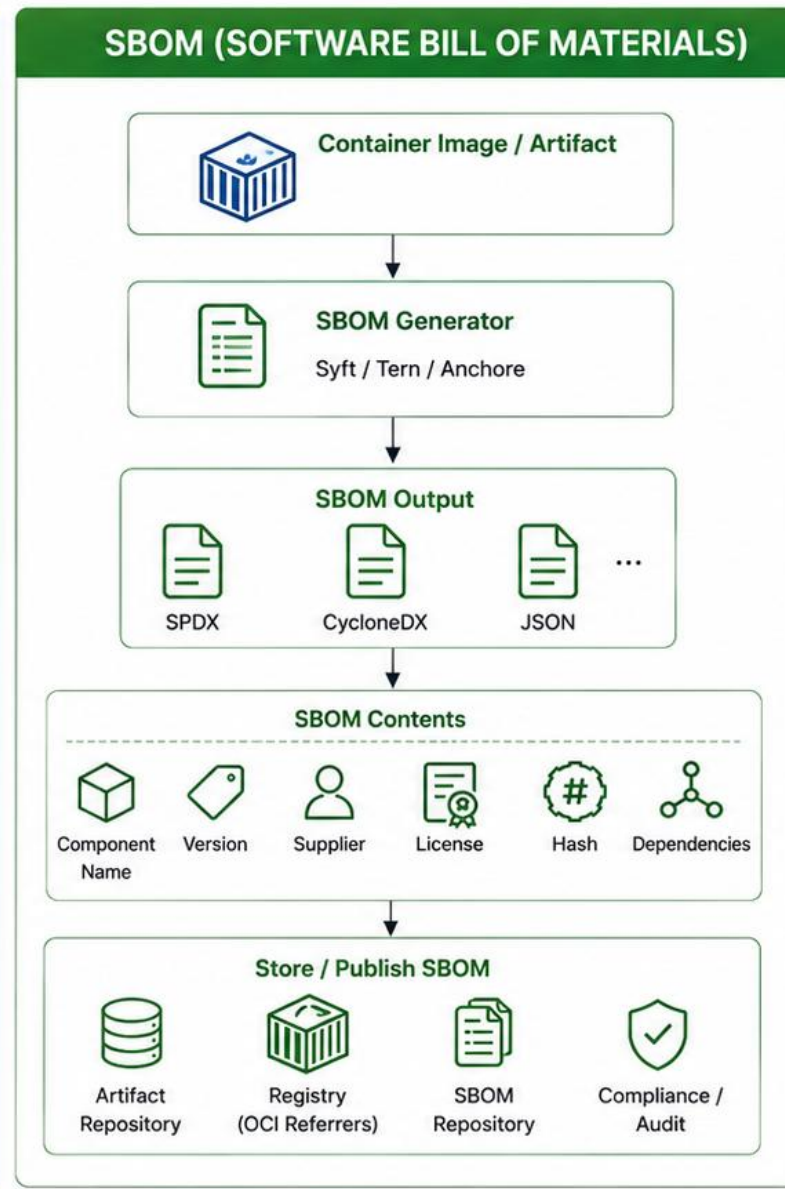
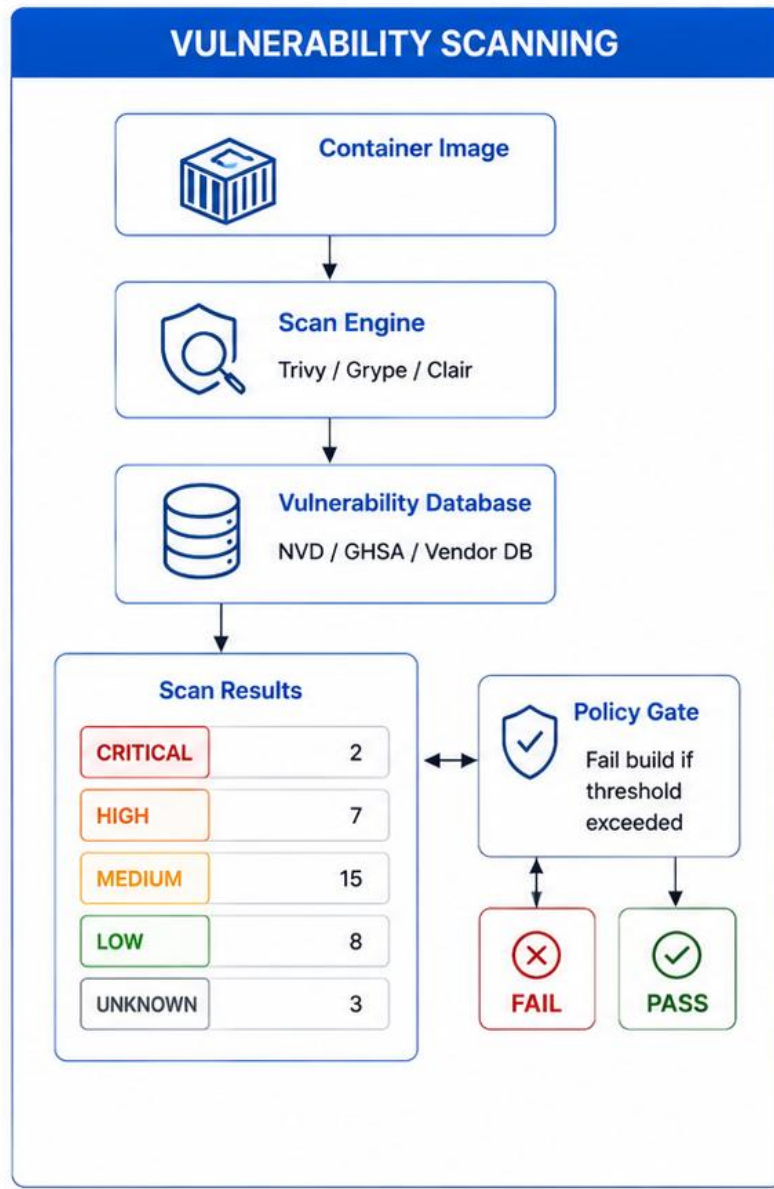
## TOOLS

- Sonatype Nexus IQ, Snyk, JFrog Xray, Dependabot, WhiteSource, Black Duck

## TOOLS

- GitHub Advanced Security, GitLab Secret Detection, Azure DevOps scanning, HashiCorp Vault

**KEY TAKEAWAY** Vulnerabilities, license risks, outdated components, and transitive dependency risk are exactly what SCA is built to surface. • API keys, tokens, passwords, certificates, and cloud credentials are all things a secrets scanner is trained to spot in code and history.



# CONTAINER IMAGE SCANNING · SHIFT-LEFT TESTING AND SECURITY · DEVSECOPS FOR MICROSERVICES

## WHAT IT DETECTS

- Known vulnerabilities (CVEs), risky base images, misconfigurations, malware, secrets baked into layers

## TOOLS

- Trivy (Aqua), Anchore Gype, Snyk Container, Red Hat Quay, Amazon ECR scanning

## INTEGRATING SECURITY AND TESTING EARLY

- **Security:** threat modeling -> secure design review -> SAST/IDE scanning -> DAST/IAST -> runtime protection
- **Testing:** requirements validation -> test planning -> unit -> integration -> UAT/E2E -> monitoring

## COST OF FIXING DEFECTS

- 1x at requirements, 5x at design, 10x at coding, 15x at testing, 30x in production

## THE SEVEN-STAGE DEVSECOPS LIFECYCLE

- Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate, with continuous feedback looping back

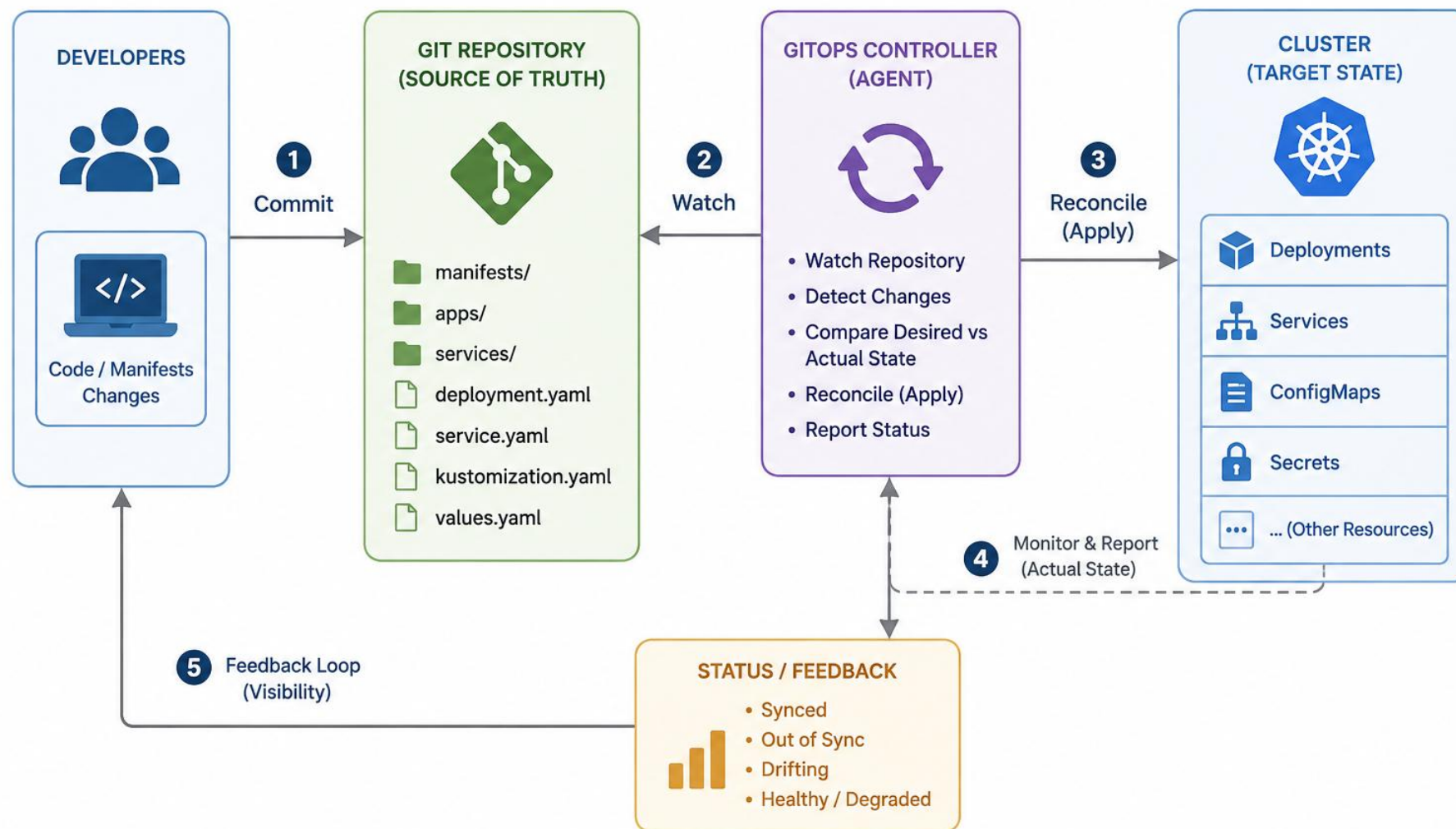
## SECURITY ACROSS THE STACK

- Application, container, platform, data, identity, and governance layers all need their own controls

**KEY TAKEAWAY** Scanning after every build and failing the pipeline on critical findings keeps vulnerable images from ever reaching the registry.

- Shift-left moves testing and security earlier into plan, design, and code, instead of waiting until test or deploy to find problems.
- Threat modeling at plan, dependency scanning at build, and runtime protection at operate turn security into a continuous thread, not a gate.

# GitOps Overview (Conceptual)



# GITHUB-BASED DEVELOPMENT WORKFLOW · GITHUB ACTIONS CONCEPTS · JENKINS ARCHITECTURE AND COMPONENTS



## THE GITHUB WORKFLOW

1. Create/Plan
2. Branch
3. Develop
4. Pull Request
5. Review & Feedback
6. Merge & Automate



## KEY BENEFITS

- Improved collaboration and code quality, faster and safer delivery, end-to-end auditability



## KEY CONCEPTS

- **Workflow:** an automated YAML-defined process that runs on events
- Event, Job, Step, Action, Runner -- the building blocks of every workflow



## EVENT TRIGGERS

- push, pull\_request, schedule, workflow\_dispatch, release, issues



## JENKINS MASTER (CONTROLLER)

- Web UI/CLI/REST API, core services (scheduling, security, plugin management), data store

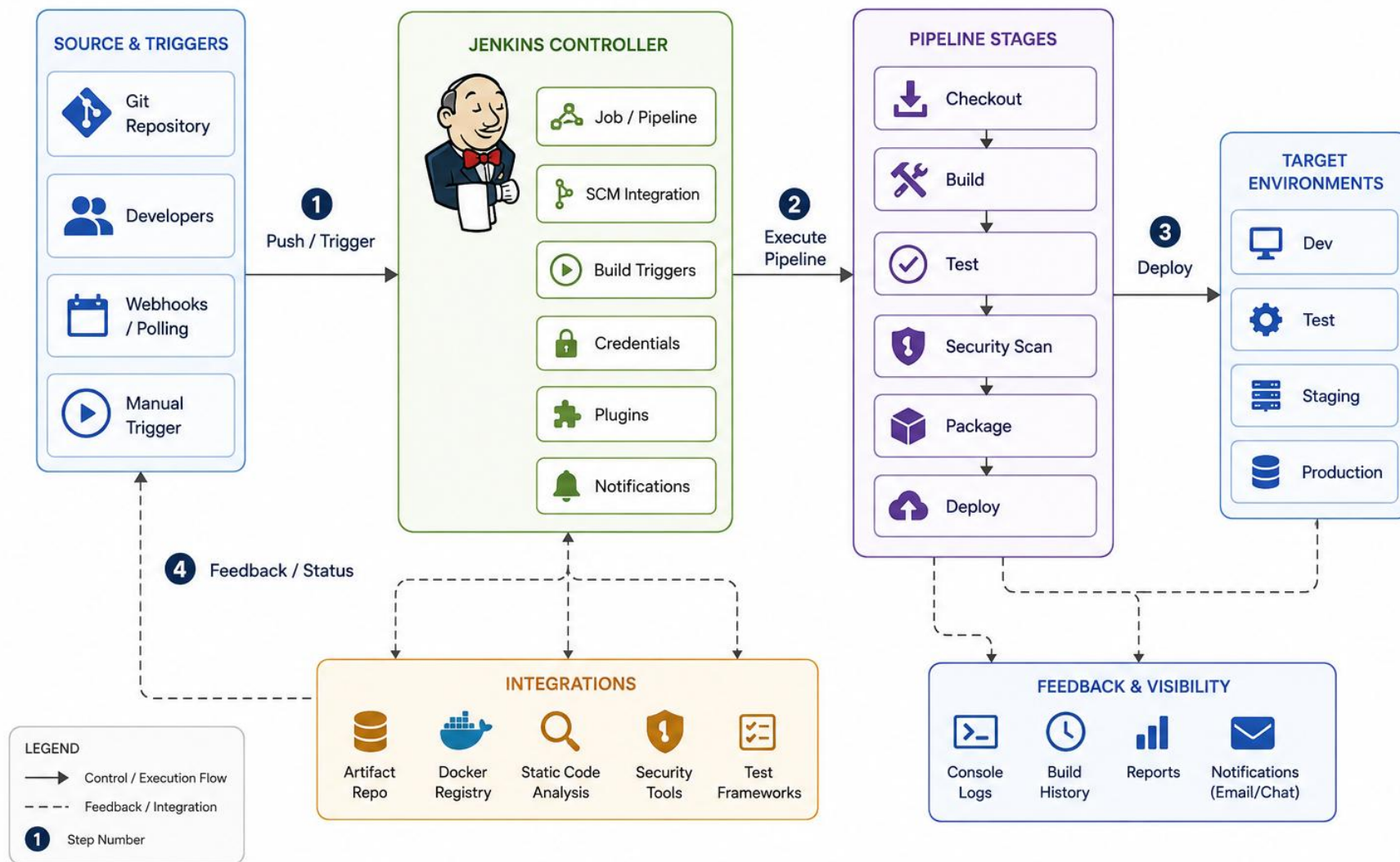


## AGENTS (BUILD NODES)

- Distributed nodes that execute builds, run tests, package artifacts, and deploy applications

**KEY TAKEAWAY** Trunk-based branching with short-lived feature branches keeps GitHub-based teams merging small, reviewed, automatically-tested changes constantly. • A push event triggers a workflow, which runs jobs in parallel, each executing its steps sequentially on a runner. • Jobs, plugins, builds, workspaces, and credentials are the core components that let a Jenkins controller coordinate work across many agents.

# Jenkins as Alternate Enterprise Pattern (awareness)



# DECLARATIVE JENKINS PIPELINES

Declarative Pipeline is a higher-level abstraction for defining CI/CD pipelines in Jenkins using a simple, structured, and readable syntax. It provides conventions, reduces boilerplate code, and improves consistency.

## WHY DECLARATIVE PIPELINES?



Simple  
and Readable



Opinionated  
Best Practices



Safer  
and Reliable



Visualization  
& Blue Ocean



Easier  
Maintenance

## KEY FEATURES

- Structured syntax with predefined sections and directives
- Built-in error handling and post conditions
- Easy parallel execution and reusability
- Extensible with shared libraries and custom steps
- Great integration with Jenkins ecosystem

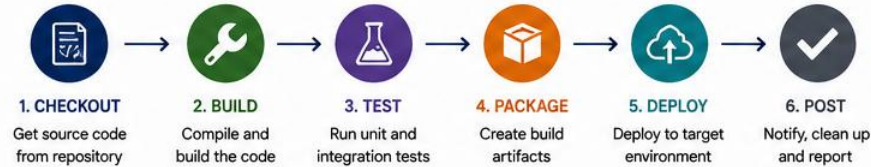
## BENEFITS

- ✓ Faster pipeline creation
- ✓ Consistency across teams
- ✓ Reduced errors and maintenance
- ✓ Better visualization (Blue Ocean)
- ✓ Supports advanced pipeline features

## COMMON DIRECTIVES

|             |                                        |
|-------------|----------------------------------------|
| agent       | Where the pipeline runs                |
| environment | Define environment variables           |
| options     | Set pipeline-wide options              |
| stages      | Group of sequential or parallel stages |
| steps       | Tasks executed in a stage              |
| post        | Actions after pipeline completion      |

## TYPICAL PIPELINE FLOW



## PIPELINE STRUCTURE

|             |                                                                      |
|-------------|----------------------------------------------------------------------|
| agent       | Defines where the pipeline runs (any, label, docker, etc.)           |
| environment | Sets environment variables for the pipeline                          |
| options     | Configures pipeline options (timeout, retry, timestamps, etc.)       |
| stages      | Defines the sequence of stages in the pipeline                       |
| steps       | Contains the steps or tasks executed in a stage                      |
| post        | Defines actions to run after the pipeline (success, failure, always) |

## EXAMPLE STAGE WITH STEPS

```
stage('Build') {
  steps {
    echo 'Building the application...'
    sh 'mvn clean package'
    archiveArtifacts artifacts: 'target/*.jar'
  }
}
```

- echo - Print messages to the console
- sh - Execute shell commands
- archiveArtifacts - Save build artifacts

## PIPELINE VISUALIZATION (BLUE OCEAN)



## SYNTAX EXAMPLE

```
pipeline {
  agent any

  environment {
    APP_NAME = 'my-app'
    MAVEN_OPTS = '-Xmx1024m'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '10'))
    disableConcurrentBuilds()
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }

  post {
    success {
      echo 'Build Successful!'
    }
    failure {
      echo 'Build Failed!'
    }
    always {
      cleanWs()
    }
  }
}
```

## BEST PRACTICES

- ✓ Keep pipelines simple and modular
- ✓ Use meaningful stage and step names
- ✓ Leverage shared libraries
- ✓ Handle errors gracefully with post
- ✓ Use environment variables for configuration
- ✓ Avoid hardcoding credentials
- ✓ Archive artifacts and logs



MD287: Spring Boot Microservices with OpenShift AI and DevOps

Module 18: CI/CD with GitHub and Jenkins | Topic 388

# MICROSERVICES PIPELINE STAGES · SOURCE CHECKOUT AND DEPENDENCY RESOLUTION · COMPILATION AND UNIT TESTING



## THE TEN PIPELINE STAGES

1. Source checkout
2. Dependency resolution
3. Compile & build
4. Unit test
5. Integration test
6. Contract test
7. Security & quality gates
8. Package artifact
9. Build container image
10. Push & deploy



## WHAT HAPPENS IN THIS STAGE

- Clone/pull latest code, validate branch and commit, read build files (pom.xml/build.gradle)
- Resolve and download dependencies, using caching and mirrors for speed



## WHAT HAPPENS IN THIS STAGE

- Source code checkout -> compile (download dependencies) -> run unit tests -> quality gate



## BEST PRACTICES

- Keep unit tests fast, isolated, and deterministic; follow Arrange-Act-Assert
- Mock external dependencies; fail the build on any compilation error or test failure

**KEY TAKEAWAY** Contract testing and security/quality gates sit deliberately before packaging, so a broken API contract or vulnerability never becomes a shipped artifact.

- Pinned dependency versions and a local cache turn a potentially flaky download step into a fast, reproducible one.
- The Arrange-Act-Assert pattern in a JUnit 5 test keeps each unit test fast, isolated, and easy to read at a glance.

# INTEGRATION AND CONTRACT TESTING · APPLICATION PACKAGING

## · CONTAINER IMAGE CREATION



### TWO COMPLEMENTARY TEST TYPES

- **Integration testing:** ensures multiple services or components work together correctly
- **Contract testing:** verifies that producers and consumers agree on the API contract



### PACT CONTRACT FLOW

- Consumer generates a contract -> provider verifies against that same contract



### PACKAGING PROCESS

1. Source code
2. Dependency resolution
3. Build
4. Package (JAR/WAR or container image)
5. Store



### TYPES OF ARTIFACTS

- **JAR/WAR:** traditional Java artifacts with an embedded server
- **Container image:** includes app, runtime, libraries, and config -- ideal for cloud-native deployment



### IMAGE CREATION PROCESS

1. Source code
2. Dockerfile
3. Build image
4. Image created
5. Push to registry



### BEST PRACTICES

- Use minimal base images (Alpine), run as non-root, use multi-stage builds
- Use .dockerignore, tag with meaningful versions, scan before publishing

**KEY TAKEAWAY** A Pact contract generated by the consumer and verified by the provider catches a breaking API change before it ever reaches production.

- An executable Spring Boot JAR's BOOT-INF/classes and BOOT-INF/lib folders bundle the compiled app and every dependency it needs.
- Each Dockerfile instruction adds a cacheable layer -- only the changed application layer needs to rebuild on most releases.

# PUBLISHING TO AN ARTIFACT REGISTRY · DEPLOYING ACROSS ENVIRONMENTS · ENVIRONMENT PROMOTION STRATEGIES



## WHY REGISTRIES MATTER

- Build image -> scan image -> push image to registry -> platform pulls image -> run container
- Registries provide authentication, authorization, scanning, audit logging, and retention policies



## DEPLOYMENT ACROSS ENVIRONMENTS

- Retrieve the built artifact once, then deploy the same version to each environment in sequence
- Configuration (URLs, credentials, replica counts) changes per environment; the artifact never does

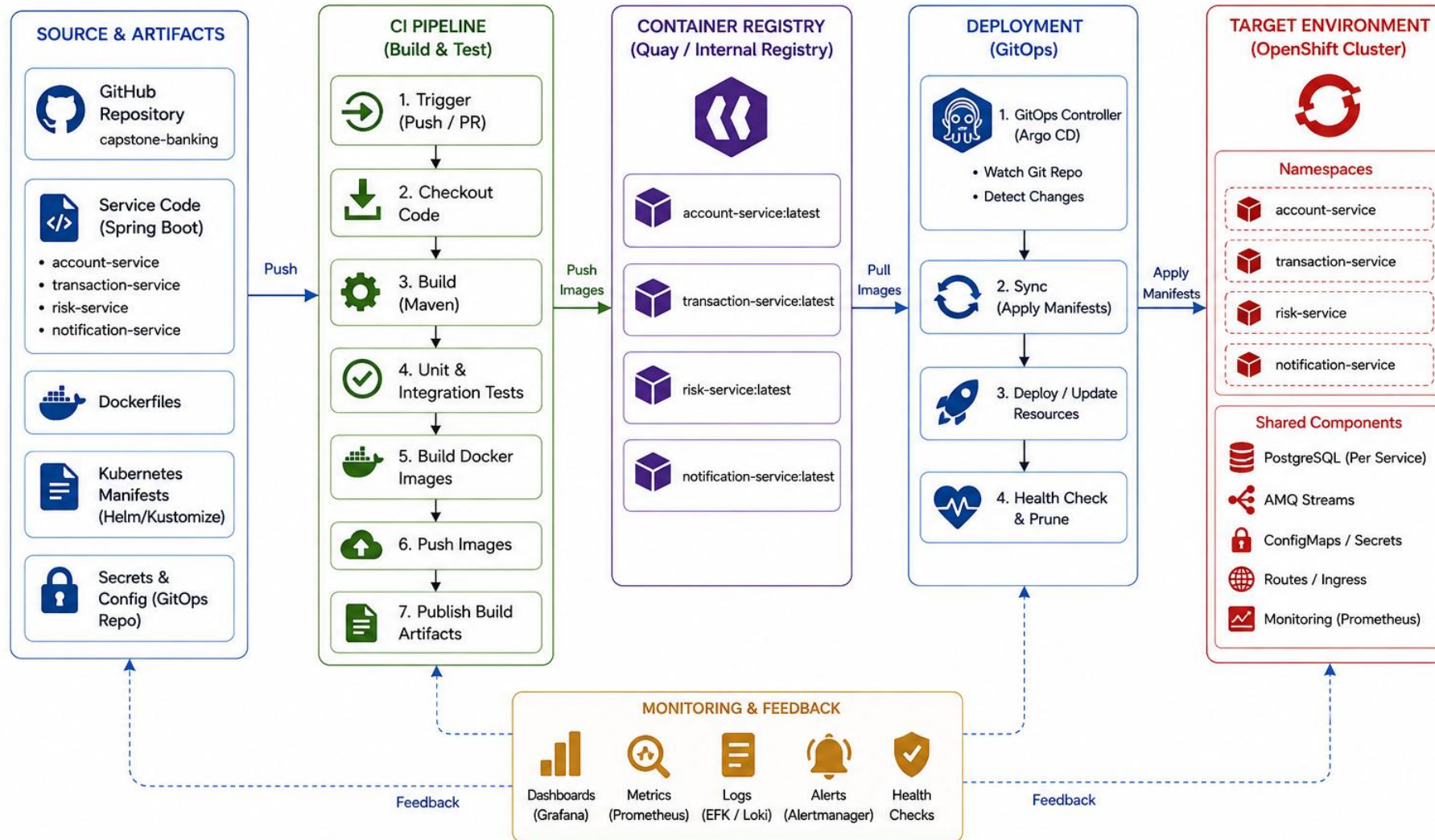


## PROMOTION FLOW

- Dev (auto-deploy on merge) -> QA/Test (automated testing) -> Staging (pre-prod validation) -> Production (manual/auto approval)
- The same artifact or image is promoted across every stage -- never rebuilt

**KEY TAKEAWAY** Immutable, versioned tags pushed to a private registry make every deployed artifact traceable back to its exact build. • Environment-specific ConfigMaps and Secrets let one immutable image behave correctly in dev, staging, and production alike. • Dev auto-deploys on merge, staging requires validation, and production requires manual or policy-gated approval.

## Lab 4 Overview - Deploy the Capstone Services



# PIPELINE VARIABLES AND CREDENTIALS · PIPELINE APPROVALS AND GOVERNANCE



## TYPES OF PIPELINE SECRETS

- GitHub tokens, registry credentials, deployment tokens, database passwords, signing keys, API keys



## BEST PRACTICES

- Store secrets in secure credential stores, never in source code or pipeline logs
- Apply least privilege and rotate credentials periodically

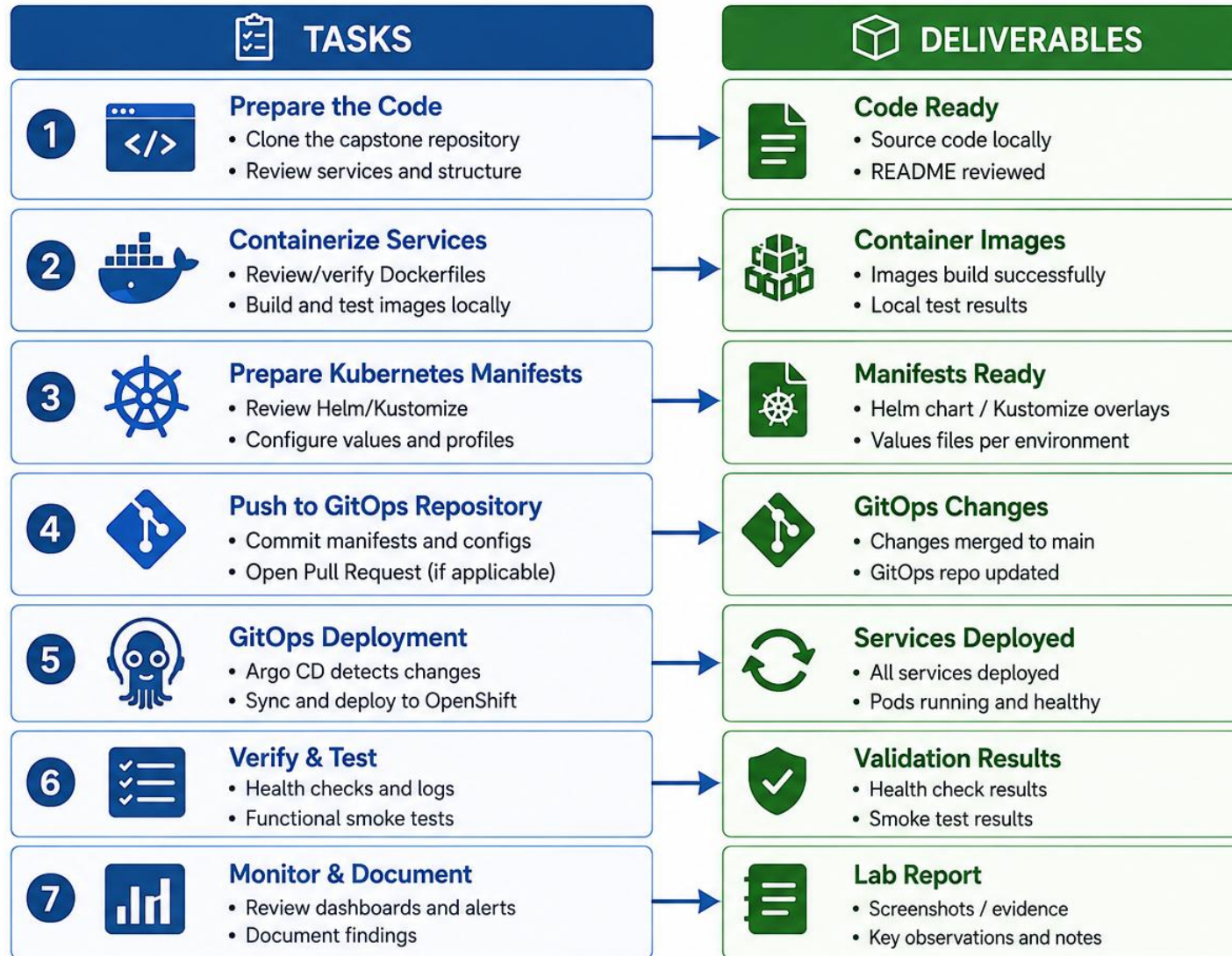


## WHY APPROVAL GATES EXIST

- Production releases often require a human decision point even in a fully automated pipeline
- Governance policies define who can approve, and what evidence (tests, scans) must be attached

**KEY TAKEAWAY** Registry tokens, deployment credentials, and database passwords for test environments all need to be injected securely, not committed. • Compliance-driven governance requires that not every pipeline stage should run unattended -- production deploys are the deliberate exception.

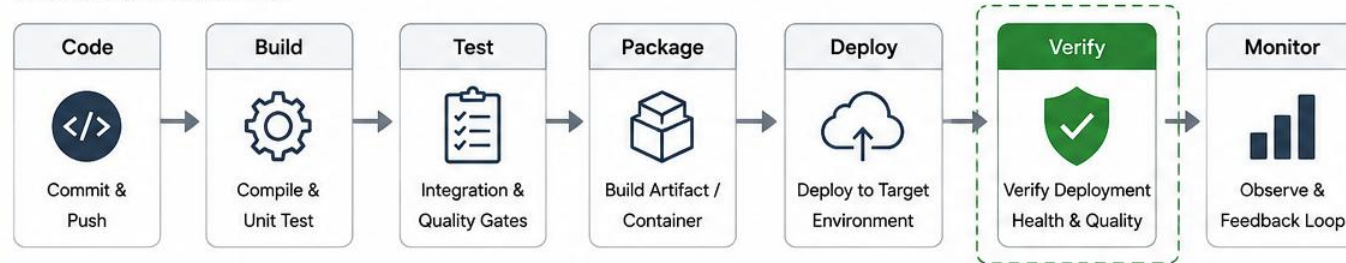
# Lab Tasks and Deliverables



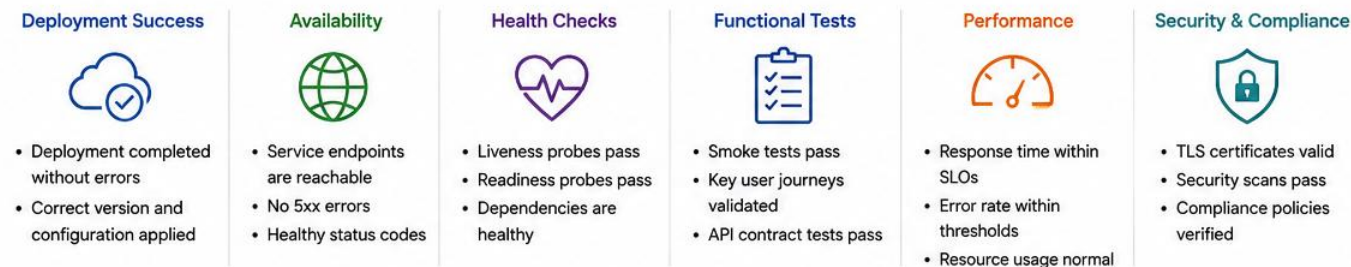
# DEPLOYMENT VERIFICATION

Confirm that what we deployed is running correctly, is healthy, and meets quality requirements.

## Where Verification Fits



## What We Verify



## Verification Techniques



## Best Practices

- ✓ Automate verification in the pipeline
- ✓ Verify in production-like environment
- ✓ Fail fast and stop promotion on failure
- ✓ Generate actionable reports and alerts
- ✓ Include rollback decision points

## Outcome

- 
- High confidence that the release is safe, functional, and ready for users.

## Verification Pillars

- Health**  
Ensure the application and infrastructure are healthy and responsive.
- Functionality**  
Validate critical business flows and API contracts work as expected.
- Performance**  
Confirm response times and throughput meet expectations.
- Security**  
Verify security controls, certificates, and access policies are enforced.
- Compliance**  
Ensure compliance checks, policies, and audit requirements pass.

Verify early. Verify often. Deliver with confidence.

Quality Releases. Reliable Services. Happy Users.

# AUTOMATED ROLLBACK STRATEGIES · PIPELINE NOTIFICATIONS AND AUDITABILITY · CI-CD PIPELINE SECURITY



## ROLLBACK OPTIONS

- Redeploy the previous version, shift traffic back to the previous environment
- Disable a feature flag, or roll forward with an emergency fix



## NOTIFICATIONS AND AUDIT TRAILS

- Notify teams of pipeline success, failure, and approval requests (Slack, email)
- Maintain audit logs of every build, deploy, and approval decision for compliance



## SECURING THE PIPELINE

- Least-privilege service accounts, short-lived credentials where supported
- Secrets scanning, dependency scanning, and image scanning built into every run
- Restrict who can modify pipeline definitions and approve production deployments

**KEY TAKEAWAY** Redeploying the previous version, switching traffic back, or disabling a feature flag are the three fastest paths back to stability.

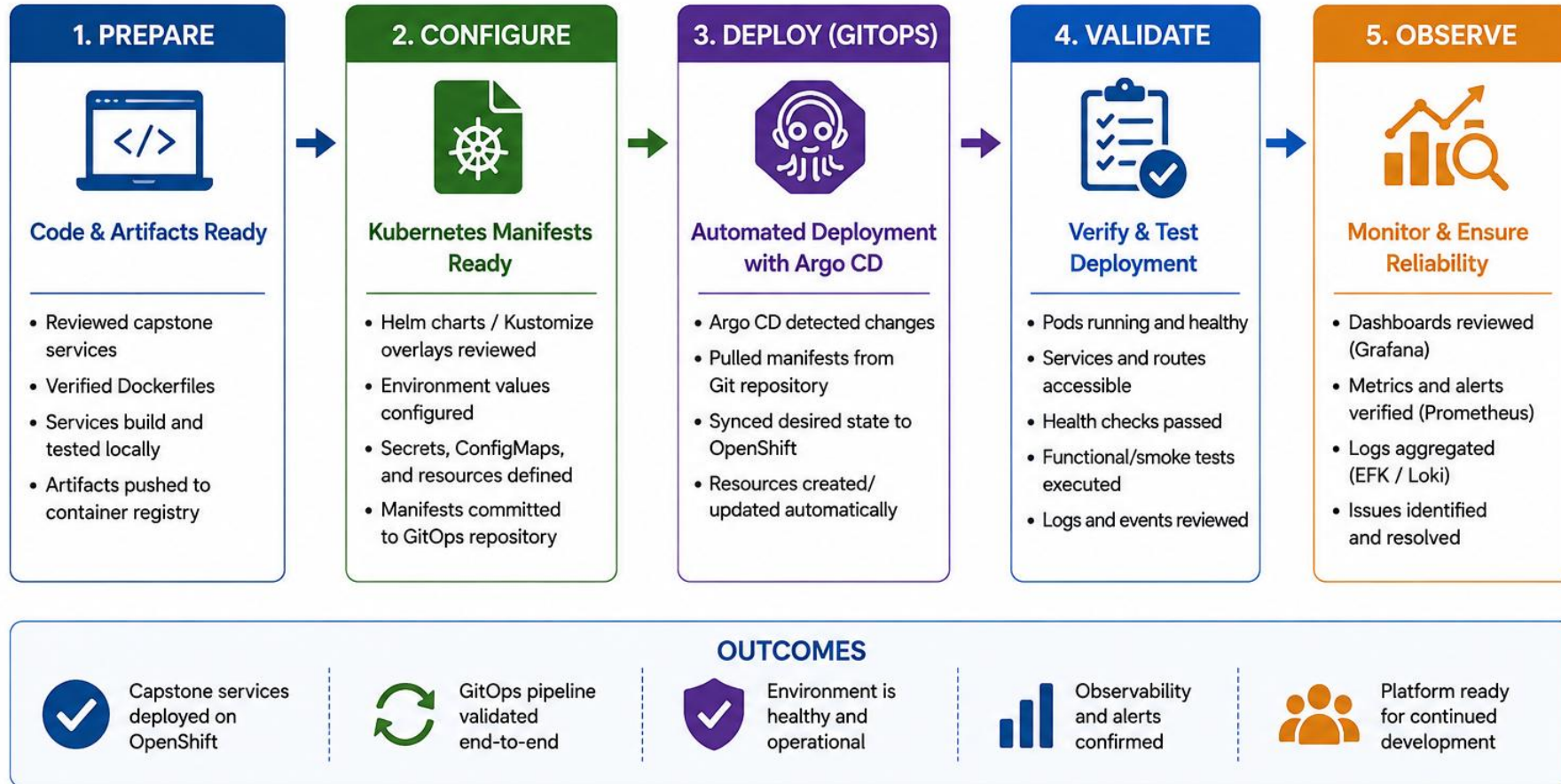
- Notifications keep teams informed in real time; audit logs let anyone reconstruct who deployed what, when, and why, after the fact.
- A compromised pipeline credential can reach source code, registries, and production deployments all at once -- it deserves top-tier protection.

# Day 4 Summary



## KEY TAKEAWAY

We built a repeatable, automated path to deploy the capstone services to OpenShift using GitOps, validated the deployment, and ensured observability and reliability.



# TROUBLESHOOTING FAILED PIPELINES

*Working backward from the first failed stage, checking its logs and inputs, resolves most pipeline failures in minutes, not hours.*



## TROUBLESHOOTING APPROACH

- Identify the first failed stage and read its full console/log output
- Check credentials, dependency availability, and environment configuration for that stage
- Reproduce locally where possible before re-running the full pipeline

**KEY TAKEAWAY** Working backward from the first failed stage, checking its logs and inputs, resolves most pipeline failures in minutes, not hours.

# Knowledge Check

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>1</b> What is GitOps and how does it work?</p>  <ul style="list-style-type: none"><li>• Git is the single source of truth</li><li>• Desired state defined in Git</li><li>• Automation reconciles actual state</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p><b>2</b> What role does Argo CD play in GitOps?</p>  <ul style="list-style-type: none"><li>• Watches Git repository</li><li>• Detects changes</li><li>• Syncs desired state to the cluster</li></ul> | <p><b>3</b> What is the flow from commit to running services?</p>  <ul style="list-style-type: none"><li>• Commit manifests to Git</li><li>• Argo CD detects changes</li><li>• Syncs to OpenShift</li><li>• Resources updated and running</li></ul>                                                                                                                                   | <p><b>4</b> What are the key benefits of GitOps?</p>  <ul style="list-style-type: none"><li>• Declarative and versioned</li><li>• Automated and consistent</li><li>• Auditable and traceable</li><li>• Safer and more reliable deployments</li></ul> | <p><b>5</b> What are Kubernetes manifests used for?</p>  <ul style="list-style-type: none"><li>• Define desired state</li><li>• Describe resources (Deployments, Services, Routes, etc.)</li><li>• Stored in Git</li></ul>                        |
| <p><b>6</b> What is a Helm chart and why is it used?</p>  <ul style="list-style-type: none"><li>• Package of Kubernetes manifests</li><li>• Reusable and parameterized</li><li>• Simplifies deployment and management</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <p><b>7</b> Where are container images stored?</p>  <ul style="list-style-type: none"><li>• In a container registry (e.g., Quay)</li><li>• Referenced by image name and tag in manifests</li></ul>      | <p><b>8</b> What is the purpose of ConfigMaps and Secrets?</p>   <p>ConfigMap      Secret</p> <ul style="list-style-type: none"><li>• ConfigMaps store non-sensitive configuration</li><li>• Secrets store sensitive data (passwords, keys, tokens)</li><li>• Injected into applications</li></ul> | <p><b>9</b> How do you verify a successful deployment?</p>  <ul style="list-style-type: none"><li>• Check pod status (Running)</li><li>• Verify services and routes</li><li>• Review logs</li><li>• Monitor metrics and alerts</li></ul>             | <p><b>10</b> What tools help you monitor and observe the platform?</p>  <ul style="list-style-type: none"><li>• Prometheus for metrics</li><li>• Grafana for dashboards</li><li>• Alertmanager for alerts</li><li>• EFK / Loki for logs</li></ul> |
| <p><b>KEY TAKEAWAYS</b></p> <div> Git is the source of truth for your infrastructure.</div> <div> Argo CD automates deployments by reconciling desired and actual state.</div> <div> Helm and manifests make deployments reusable and consistent.</div> <div> ConfigMaps and Secrets keep configuration secure and portable.</div> <div> Observability ensures reliability and helps resolve issues quickly.</div> |                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                      |

# EXERCISE 4.3 — TRACE A PIPELINE STAGE OWNERSHIP MAP

*Module 11 checkpoint C — Who owns scan / SBOM / sign / promote.*



## OBJECTIVE

- Who owns scan / SBOM / sign / promote
- 15 min · worksheet checkpoint
- **Worksheet:** labs/day-04/lab4/starter/pipeline/OWNERSHIP.md



## DO THIS

1. Open labs/day-04/lab4/starter/pipeline/OWNERSHIP.md and fill scan / SBOM / sign / promote
2. CRITICAL CVE fails the gate. SBOM is the ingredients list. Unsigned images must not run
3. Participants execute and verify — they do not manage signing keys
4. GitOps is conceptual. Do not install Jenkins or GitHub Actions runners



## EXPECTED RESULT

- A completed ownership map. Compare with labs/day-04/lab4/solution/pipeline/OWNERSHIP.md when asked.
- Fill OWNERSHIP.md in Lab 4 before run-pipeline-locally.ps1. Jenkins is awareness only.

**KEY TAKEAWAY** A completed ownership map. Compare with labs/day-04/lab4/solution/pipeline/OWNERSHIP.md when asked.

## LAB 4

# Lab 4 — Deploy the Capstone Services

Services observable, containerized, and deployed via a completed CI/CD pipeline.



# LAB 4 — DEPLOY THE CAPSTONE SERVICES — OVERVIEW

*Day 4 lab: what you build and how it grows the capstone.*



## WHAT YOU WILL FINISH WITH

- Multi-stage Containerfiles that run as user md287, not root
- Account on 8081 and Transaction on 8082 from Docker images (Compose, then OpenShift)
- Liveness and readiness probes hitting Actuator
- ConfigMaps for URLs, Secrets for passwords and the classroom JWT applied on OpenShift
- Resource requests and limits on the OpenShift Deployments
- Working Routes on the classroom cluster
- Logs that still print correlationId= and never print Authorization
- **A completed prepared pipeline:** scan gate, SBOM, signature verify
- A version bump (1.0.0 → 1.0.1) and oc rollout undo back to 1.0.0

**KEY TAKEAWAY** Lab 4 builds Deploy the Capstone Services -- the working code behind today's concepts, and the next layer of the banking capstone.

# LAB 4 — DEPLOY THE CAPSTONE SERVICES — BEFORE YOU START

*Check these before the lab clock starts.*

 **KNOWLEDGE YOU NEED (FROM DAY 4)**

| Day 4 idea                       | How it appears in this lab                                                                                  |
|----------------------------------|-------------------------------------------------------------------------------------------------------------|
| Liveness vs readiness vs startup | HTTP GET /actuator/health/liveness and /actuator/health/readiness. Startup probe gives Spring time to boot. |
| Correlation ID                   | Header X-Correlation-Id still lands in logs as correlationId=. That is the classroom trace handle.          |
| Metrics                          | Compose exposes health,info,metrics. Call /actuator/metrics with a JWT. Health stays public.                |
| Non-root image                   | USER md287 in the Containerfile. OpenShift also sets runAsNonRoot: true.                                    |
| Config vs secret                 | JDBC URL in a ConfigMap. DB password and JWT secret in a Secret.                                            |
| Immutable tags                   | Promote 1.0.0 / 1.0.1, not :latest.                                                                         |
| Pipeline gates                   | CRITICAL CVE → fail. SBOM is the ingredients list. Unsigned images must not run.                            |

**KEY TAKEAWAY** Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

# LAB 4 — DEPLOY THE CAPSTONE SERVICES — LAB STEPS

*7 steps, worked in order.*

## LAB STEPS

- Step 1 — Confirm the Lab 3 images you will ship
- Step 2 — Finish the Containerfiles (non-root)
- Step 3 — Exercise 4.2 recap (layered image)
- Step 4 — Run the stack and prove probes, metrics, and correlation

## LAB STEPS (CONTINUED)

- Step 5 — Log in, apply manifests, and push images (OpenShift — required)
- **Step 6 — Prepared pipeline:** SBOM, signature, scan gate
- Step 7 — Deploy a new version and roll it back (OpenShift — required)

**KEY TAKEAWAY** Work the steps in order -- each one builds on the last.

# LAB 4 — DEPLOY THE CAPSTONE SERVICES — SUCCESS CRITERIA

*You are done when every box below is ticked.*

## SUCCESS CRITERIA

- Account and Transaction images build from Lab 3 solution context
- Container process is not root (docker run --rm --entrypoint id ...)
- **Compose stack:** liveness, readiness, /actuator/metrics, /actuator/info
- JWT still required on business APIs; health stays public
- Correlation id appears in logs; no Authorization text

## SUCCESS CRITERIA (CONTINUED)

- oc whoami works; manifests applied in the assigned project (not 00-namespace.yaml)
- Images pushed; pods Ready; Account Route readiness 200
- **Pipeline:** CRITICAL scan fails the gate; SBOM reviewed; signature command reviewed
- oc rollout undo restored 1.0.0 (history shows more than one revision)
- Ownership map for Exercise 4.3 filled

**KEY TAKEAWAY** Tick every criterion before moving on -- the next lab builds directly on this one.

*"Services observable, containerized, and deployed via a completed CI/CD pipeline."*

DAY 4 COMPLETE

# Observability, Containers, OpenShift, and CI/CD

Day 4 complete — Services observable, containerized, and deployed via a completed CI/CD pipeline.

